SYSTEM DESIGN MASTERY

The Complete Developer's Guide

From Internship to Principal Engineer

35 Deep-Dive Lectures • Real Systems • Production-Level Knowledge

Scalability • Caching • Databases • Microservices • Kubernetes • and more

How to Use This Book

This book is designed for every developer — whether you just wrote your first API or you've been building systems for a decade. Each lecture builds on the last, introducing concepts progressively with real-world examples, deep explanations, and interview-ready insights.

Who is this book for?

- Interns and junior developers who want to understand how large systems work
- Mid-level engineers preparing for system design interviews
- Senior engineers who want to refresh fundamentals and discover new patterns
- Architects designing distributed systems at scale

How to read this book

1. Read sequentially — each lecture assumes knowledge from the previous ones
2. Pay extra attention to the sections marked with ⭐ — these are interview gold
3. Look for the 💡 Analogy boxes — they make abstract concepts stick
4. Re-read the Key Takeaways at the end of each lecture to reinforce learning
5. Try to relate every concept to systems you use daily: Instagram, Amazon, WhatsApp

> 🔍 **A note on depth:** This book does not just scratch the surface. Every concept is explained at production level — the way it works inside real systems at companies like Google, Netflix, Amazon, and Uber. The goal is deep understanding, not just awareness.

LECTURE 1: What is a System?

Before you can design systems, you need to understand what a system actually is — not just as a buzzword, but as a precise mental model you can apply to everything you build.

1. The Universal Definition

A system is a collection of components that work together to achieve a specific goal. This definition sounds simple, but it contains enormous depth.

> **Real-World Analogy:** Think of the human body. Your heart, lungs, brain, and blood vessels are individual components. Alone, none of them can sustain life. Together, they form a system that does. Software systems follow the same principle.

Every system — no matter how complex — can be described by three elements:

- Input — something the system receives (a user request, a sensor reading, a file)
- Processing — what the system does with that input (business logic, computation, transformation)
- Output — what the system produces (a response, a stored record, an event)

Concrete Example: ATM Machine

Component	Role
Card + PIN	Input — user provides identity
Bank verification	Processing — validates account, checks balance
Cash dispensed	Output — the result the user receives
Transaction logged	Side effect — system records what happened

2. A Software System in Practice

In software engineering, a system is a combination of services, servers, databases, APIs, and networks that work together to deliver functionality to users. The components can be physical machines, virtual services, or even simple functions — but they must collaborate.

When you open Amazon.com, this is what happens behind the scenes:

6. Your browser sends a request to Amazon's servers
7. DNS resolves amazon.com to an IP address
8. A load balancer distributes your request to one of hundreds of servers
9. An application server processes the request and fetches product data
10. That data may come from cache (Redis) or a database (DynamoDB)
11. The response travels back through the network to your browser

12. Your browser renders the HTML and you see the homepage

All of that happened in under 200 milliseconds. That's a system working correctly.

3. The Core Components of Any System

Client

The client is the user-facing layer — the part users interact with directly. It sends requests and displays results. Clients are intentionally kept lightweight; they don't run heavy business logic or store large amounts of data.

Examples: Web browsers, mobile apps, desktop applications, IoT sensors, CLI tools.

> **Pro Tip:** In system design interviews, always clarify who the clients are. Is it a mobile app? A third-party API? An internal microservice? The answer shapes your entire design.

Server

A server is a machine or process that listens for requests, executes business logic, and returns responses. It's the brain of your system. In modern architectures, 'server' is increasingly abstract — it might be a containerized service, a serverless function, or a cloud-managed runtime.

What servers do: authenticate requests, apply business rules, query databases, call other services, and return structured responses.

Database

The database is the system's memory. Without it, every restart wipes the slate clean. Databases come in many flavors — relational (MySQL, PostgreSQL), document (MongoDB), key-value (Redis), and more. Choosing the right database is one of the most important decisions in system design, and we'll spend several lectures on it.

Network

The network is the invisible highway connecting every component. It operates using protocols — rules for how data is formatted and transmitted. The most important protocols you'll encounter are HTTP, HTTPS, TCP/IP, WebSocket, and gRPC. Network reliability, latency, and bandwidth are critical constraints in distributed systems.

Subsystems

Large systems are divided into smaller, specialized subsystems. This is not just organizational preference — it's an engineering necessity. When one subsystem fails, the others continue. When traffic to one subsystem spikes, only that piece needs to scale.

4. Monolithic vs. Distributed Systems

Early systems put everything in one place — one codebase, one server, one database. This is called a monolith. It's simple to build and deploy, which is why startups often begin here.

As systems grow, the monolith becomes a liability. A single bug can crash everything. A single team must coordinate all changes. Scaling requires scaling the entire system, even if only one feature is under load.

Modern systems are distributed — multiple independent services collaborating over a network. This introduces complexity (network failures, data consistency, service discovery) but enables massive scale, independent deployment, and fault isolation.

5. System Boundaries

Every system has a boundary — the line between what is yours and what is external. Inside the boundary: your services, your databases, your business logic. Outside: payment gateways, mapping APIs, email providers, SMS services.

Understanding boundaries matters enormously in system design. It defines your attack surface, your points of failure, your SLA dependencies, and your integration contracts.

> ⍰ **Interview Insight:** In an interview, when asked to design a system, one of the first things you should establish is the system boundary. What's inside? What's external? This immediately shows the interviewer that you think at an architectural level.

6. System Failures Are Normal

Every system eventually fails. Hardware breaks. Networks partition. Code has bugs. The mark of a well-designed system is not that it never fails, but that it fails gracefully — it degrades predictably, recovers automatically, and limits the blast radius of any single failure.

We will explore failure handling in depth in Lectures 16 and 27. But internalize this now: designing for failure is not pessimism — it is professional engineering.

> ⍰ **Key Takeaways:** A system is components working together. Every system has inputs, processing, and outputs. Systems have boundaries between internal and external components. Modern systems are distributed. Failures are expected — design for them.

LECTURE 2: Basic Building Blocks — Client, Server, Network, DNS

Every internet application — no matter how complex — is built on four fundamental building blocks. Understanding these deeply means understanding how requests travel from your fingertips to a server halfway around the world and back in milliseconds.

1. The Client

A client is any application that makes requests to a server. Clients are consumers of services. They initiate communication, receive responses, and render information for users.

Clients are intentionally kept thin. They should not contain critical business logic, sensitive data, or heavy computation. Why? Because clients run on devices you don't control. A web browser or mobile app can be inspected, modified, or spoofed. So we push logic to the server.

Types of Clients

- Web browsers — Chrome, Firefox, Safari, Edge
- Mobile applications — iOS apps (Swift), Android apps (Kotlin)
- Desktop applications — Electron apps, native apps
- Command-line tools — curl, custom CLIs
- IoT devices — smart sensors, embedded systems
- Other services — microservices calling each other (server-to-server clients)

> **Pro Tip:** In a distributed system, services act as clients of other services. Your Order Service is a client of your Payment Service. Keep this in mind — the client/server model is recursive.

2. The Server

A server is a machine (physical or virtual) running software that listens for incoming requests, processes them, and returns responses. The server is where your business logic lives.

When you write a .NET Core Web API, a Node.js Express app, or a Java Spring Boot service, you are building a server. The server is responsible for:

- Receiving and parsing incoming requests
- Authenticating and authorizing the caller
- Executing business logic
- Reading from and writing to databases
- Calling downstream services
- Formatting and returning responses

Stateless vs. Stateful Servers

This distinction is crucial in system design. A stateless server does not remember anything between requests. Every request is handled independently, as if the server has never seen this client before. This makes stateless servers trivially easy to scale — just add more of them. HTTP APIs are naturally stateless.

A stateful server maintains context between requests — a WebSocket connection, a gaming session, a streaming pipe. Stateful servers are harder to scale because you must ensure subsequent requests from the same client reach the same server, or synchronize state across servers.

> **Golden Rule:** Design your services to be stateless wherever possible. Push state into dedicated storage layers (databases, caches, session stores). This single decision makes horizontal scaling straightforward.

3. The Network

The network is the invisible infrastructure that connects everything. Without it, your client cannot reach your server, your server cannot reach its database, and your services cannot call each other.

Understanding networking is not optional for a systems designer. You don't need to be a network engineer, but you must understand the key protocols and their implications.

Protocol	Layer	Purpose	Key Characteristic
IP	Network	Addressing — every device gets an IP	Unreliable, best-effort delivery
TCP	Transport	Reliable data transmission	Guarantees delivery and order; slower
UDP	Transport	Fast data transmission	No guarantees; used for video, gaming
HTTP/1.1	Application	Web communication	Text-based, one request per connection
HTTP/2	Application	Optimized web communication	Multiplexed, binary, compressed headers
HTTPS	Application	Encrypted web communication	TLS layer on top of HTTP
WebSocket	Application	Persistent bidirectional communication	Real-time apps — chat, live feeds
gRPC	Application	High-performance service-to-service	Protocol buffers, strongly typed

Latency Across the Network

Physical distance matters. Data travels at roughly the speed of light through fiber optic cables, but network hops, routing, and protocol overhead add latency. This is why global companies place servers in multiple geographic regions — to serve users from nearby machines.

Typical latency figures every engineer should know:

- RAM access: ~100 nanoseconds

- SSD read: ~100 microseconds
- Same datacenter network: ~1 millisecond
- Cross-country (US): ~40–80 milliseconds
- Transatlantic (US to Europe): ~80–120 milliseconds
- US to India: ~200–300 milliseconds

> ▯ **Interview Insight:** Knowing these latency numbers cold impresses interviewers and helps you make principled tradeoffs. Why use a CDN? Because cross-continent latency is 200ms but same-country is 5ms.

4. DNS — The Internet's Phonebook

DNS (Domain Name System) is the system that translates human-readable domain names into machine-readable IP addresses. Humans remember 'google.com'; computers need '142.250.183.14'.

The Full DNS Lookup Chain

13. Browser cache — if visited recently, the IP is cached locally
14. Operating system cache — OS maintains its own DNS cache
15. Router cache — your home/office router may cache DNS results
16. ISP's DNS resolver — your internet provider's DNS server
17. Root nameservers — the top of the DNS hierarchy (there are 13 root server clusters globally)
18. TLD nameservers — responsible for .com, .org, .in, etc.
19. Authoritative nameserver — the DNS server for that specific domain
20. Response travels back and is cached at each level for the TTL duration

This entire lookup typically takes 10–50ms, and most requests skip most steps due to caching.

> ▯ **Real-World Analogy:** DNS is like a distributed, hierarchical phonebook. Instead of one central book, the lookup system is replicated across thousands of servers worldwide, making it incredibly resilient and fast.

5. A Complete Request Journey

Let's trace exactly what happens when you type 'amazon.com' in your browser and press Enter. This is the kind of end-to-end thinking that distinguishes senior engineers.

21. Browser checks DNS cache — not found (first visit)
22. OS checks its DNS cache — not found
23. DNS query sent to ISP's resolver
24. Resolver queries root nameserver → gets TLD (.com) nameserver address
25. Resolver queries .com nameserver → gets Amazon's authoritative nameserver address
26. Resolver queries Amazon's nameserver → gets the IP address
27. Browser opens TCP connection to that IP (three-way handshake: SYN → SYN-ACK → ACK)
28. TLS handshake for HTTPS (verifies certificate, establishes encryption)

29. Browser sends HTTP GET request: GET / HTTP/1.1 Host: amazon.com
30. Amazon's load balancer receives the request, forwards to an application server
31. Application server fetches data from cache/database, generates response
32. Response travels back: HTML, CSS, JavaScript, image URLs
33. Browser parses HTML, fetches additional resources in parallel
34. Page renders — user sees Amazon homepage

This whole sequence completes in under 500ms on a good connection. That's the astonishing efficiency of modern internet infrastructure.

> **Key Takeaways:** Clients initiate; servers process; networks transport; DNS resolves. Stateless servers scale easily. Network latency is real — geography matters. DNS is hierarchical and heavily cached.

LECTURE 3: Scalability — Vertical vs. Horizontal Scaling

Scalability is the ability of a system to handle growing load without degrading performance. It is not an afterthought — it's a first-class design concern for any system expected to grow.

> **Real-World Analogy:** Imagine a restaurant. When a few customers come, one chef handles everything. As the restaurant becomes popular, you have two options: hire a super-chef who cooks faster (vertical scaling) or hire more chefs (horizontal scaling). The second approach is how the internet was built.

1. Why Scalability Matters

Small systems often work fine with naive designs. But as load increases, poorly designed systems fail in predictable ways: databases crash, servers overload, response times balloon, and users leave.

Famous scaling failures you should know:

- Pokémon Go (2016): servers crashed on launch because demand was 50x the estimate
- Twitter's Fail Whale era (2007–2012): the infamous 'Twitter is over capacity' error
- Healthcare.gov (2013): US government healthcare site couldn't handle launch traffic

These weren't caused by bad engineers — they were caused by insufficient scalability planning.

2. Vertical Scaling — Scaling Up

Vertical scaling means making your existing machine more powerful. More CPU cores, more RAM, faster SSD, better network interface. The same server handles more work because it has more raw capacity.

Resource	Before	After Vertical Scale
CPU	4 cores	64 cores
RAM	8 GB	512 GB
Storage	500 GB SSD	10 TB NVMe SSD
Network	1 Gbps	100 Gbps

When Vertical Scaling Makes Sense

- Early-stage startups — complexity overhead of distributed systems isn't justified
- Databases — relational databases often scale vertically longer than you'd expect
- Monolithic applications — when splitting into services isn't yet warranted
- In-memory computation workloads that need large RAM

The Hard Limits of Vertical Scaling

- Hardware ceiling — servers can't grow infinitely; the largest cloud instances have limits
- Cost curve — high-end servers are exponentially more expensive per unit of performance
- Single point of failure — if that one server crashes, your entire system is down
- Maintenance windows — upgrading physical hardware requires downtime

> ⚠ **Warning:** Never rely solely on vertical scaling for a system that needs high availability. A single server, no matter how powerful, is a single point of failure.

3. Horizontal Scaling — Scaling Out

Horizontal scaling means adding more machines to your pool. Instead of one powerful server, you run many standard servers in parallel. Each server handles a portion of the traffic.

This is the dominant scaling strategy for modern internet systems. Netflix, Google, Amazon, and Meta all run horizontal scaling as their primary approach to capacity.

The Load Balancer — Making Horizontal Scaling Work

When you have multiple servers, you need something to distribute traffic among them. That's the load balancer. Clients send requests to the load balancer, which forwards each request to one of the available servers using a distribution algorithm.

We'll cover load balancing deeply in Lecture 5, but conceptually: the load balancer is what turns multiple independent servers into a single logical service from the client's perspective.

Horizontal Scaling in Action

Imagine your API handles 10,000 requests per second on a single server at 80% capacity. Traffic doubles. Options:

- Vertical: upgrade to a 2x more powerful server — expensive, temporary, single point of failure
- Horizontal: add a second identical server, load balance between them — scalable, fault tolerant, cost-effective

If traffic doubles again, add two more servers. This approach scales linearly — and with cloud providers like AWS, Azure, and GCP, you can do it automatically.

4. Auto Scaling — The Cloud Superpower

Modern cloud platforms allow automatic scaling based on real-time metrics. When CPU usage exceeds 70%, the platform automatically provisions new server instances. When traffic drops at 3 AM, instances terminate to save cost.

Auto Scaling Flow

35. Monitoring system detects CPU at 85%

36. Scaling policy triggers: 'add 2 more servers'
37. Cloud platform provisions new instances from your machine image
38. New instances start, pass health checks, and register with the load balancer
39. Load balancer begins routing traffic to new instances
40. Total capacity increases — load returns to acceptable levels

This entire process can take 2–5 minutes with modern infrastructure. AWS Auto Scaling Groups, Azure VMSS, and GCP Managed Instance Groups all provide this capability.

5. The Database Scaling Challenge

Servers are relatively easy to scale horizontally — they're stateless, so any server can handle any request. Databases are harder because they're stateful. Multiple database instances need to stay in sync.

The primary strategies for database scaling are:

- Read replicas — copies of the database that handle read queries (Lecture 7)
- Sharding — splitting data across multiple databases (Lecture 8)
- Caching — serving frequent reads from fast in-memory stores (Lecture 9)

These strategies don't replace each other — mature systems use all three.

6. Scaling Bottlenecks — Where Systems Break

As you scale, new bottlenecks emerge. This is Amdahl's Law in practice: you can't speed up a system faster than its slowest serial component. Common bottlenecks:

- Database write throughput — eventually your primary DB can't accept more writes
- Network bandwidth — data transfer rates between services
- Disk I/O — storage read/write speeds
- CPU saturation — computation-bound tasks fill all cores
- Memory — insufficient RAM causes swapping, which is catastrophically slow

> ⍰ **Interview Insight:** In system design interviews, when you add a component to handle scale, always ask yourself: 'Have I just moved the bottleneck instead of eliminating it?' Thinking in bottlenecks shows architectural maturity.

7. Measuring Scalability

Scalability is not binary — it's measured. Key metrics:

Metric	Definition	Target for large systems
RPS (Requests/sec)	How many requests the system processes per second	10,000 – 1,000,000+

Metric	Definition	Target for large systems
Latency at P99	Response time for the slowest 1% of requests	< 500ms
Throughput	Data volume processed per unit time	System-specific
Availability	Percentage of time the system is up	99.99% (52 min downtime/year)
Error rate	Percentage of requests that fail	< 0.1%

Key Takeaways: Vertical scaling = bigger machine. Horizontal scaling = more machines. Horizontal scaling + load balancing is the internet's dominant model. Databases are harder to scale than stateless services. Auto scaling matches capacity to demand automatically.

LECTURE 4: Latency, Throughput & Performance Metrics

Performance is the dimension most users experience directly. A system that works correctly but responds slowly is a failed system. Understanding performance metrics precisely is the foundation of performance engineering.

1. Latency — Time per Request

Latency is the time elapsed from when a client sends a request to when it receives a complete response. It is measured in milliseconds (ms) for most web systems, microseconds (µs) for in-memory operations, and nanoseconds (ns) for CPU operations.

Latency is not a single number — it's a distribution. Most requests are fast; some are slow; a few are very slow. This is why we measure percentiles, not just averages.

Types of Latency

- Network latency — time for data to travel across the network (affected by geography and routing)
- Processing latency — time the server spends executing business logic
- Database latency — time to execute queries and return results
- Queue latency — time a request spends waiting to be processed
- Disk I/O latency — time to read from or write to storage

2. Throughput — Requests per Second

Throughput measures how many requests a system can process per unit of time — typically expressed as RPS (requests per second) or TPS (transactions per second). While latency measures the experience of individual requests, throughput measures the system's aggregate capacity.

> **Real-World Analogy:** Back to our pizza shop: latency is how long it takes to make one pizza. Throughput is how many pizzas the shop produces per hour. A shop that makes each pizza slowly but has 20 ovens running in parallel can still have high throughput.

The Relationship Between Latency and Throughput

Latency and throughput are related but not the same. In many systems:

- Increasing throughput (more concurrent requests) increases latency — servers have to queue work
- Reducing latency often improves throughput — faster processing means more capacity
- At some point, adding more load doesn't increase throughput — it just increases queue depth and latency

This 'saturation point' is a critical design boundary. Design your system to operate well below saturation under normal conditions, with headroom for traffic spikes.

3. Percentile Latency — P50, P95, P99

The average is a dangerous metric for latency. If 99% of requests complete in 10ms but 1% take 5000ms, the average might be ~60ms — misleadingly good. The users experiencing 5000ms latency are your unhappiest users, and they're invisible in the average.

Percentiles reveal the true picture:

Percentile	Meaning	What it tells you
P50 (median)	50% of requests complete at or below this time	Typical user experience
P90	90% of requests complete at or below this time	Near-worst case experience
P95	95% of requests complete at or below this time	Most users' experience
P99	99% of requests complete at or below this time	Tail latency — the worst 1%
P99.9	99.9% of requests complete at or below this time	Used for extremely strict SLAs

Example real measurements for a production API:

Metric	Value
P50	45ms
P90	120ms
P95	280ms
P99	850ms
P99.9	3,200ms

This tells you most users have a great experience, but 1 in 1,000 requests is very slow. This is a common pattern — often caused by garbage collection pauses, cold cache misses, or database lock contention.

> ⍰ **Interview Insight:** Always define and monitor tail latency (P99) in production. Amazon famously discovered that optimizing P99 had more business impact than optimizing P50, because slow pages disproportionately affect user satisfaction.

4. Performance Metrics Every Engineer Should Know

Metric	Definition	Good/Bad
Response time	End-to-end time from client perspective	< 200ms (good), > 1s (bad)
RPS (Requests/sec)	Server requests processed per second	Higher is better
Error rate	Percentage of requests that fail	< 0.1% (good), > 1% (investigate)
CPU utilization	Percentage of CPU in use	60-70% normal, >85% danger zone
Memory utilization	RAM in use as percentage of total	> 90% risks OOM kills
Disk I/O wait	Time processes spend waiting for disk	> 20% indicates disk bottleneck
Network bandwidth	Data transferred per second	Must be below link capacity
Cache hit rate	Percentage of reads served from cache	> 90% is target

5. Reducing Latency — The Engineering Toolkit

Caching

The single most powerful latency reduction technique. Serving data from RAM (microseconds) instead of a database query (milliseconds) can reduce latency by 100x. We'll dedicate two full lectures to caching strategies.

Database Query Optimization

Slow queries are one of the top sources of latency in production systems. Common improvements: add indexes on frequently queried columns, rewrite N+1 queries as joins, use query explain plans to identify table scans, and denormalize data to reduce join complexity.

Content Delivery Networks (CDN)

Serving static assets (images, JavaScript, CSS, videos) from a CDN node physically close to the user reduces network latency by 10-100x. A user in Mumbai fetching from a Mumbai CDN node has ~5ms latency; fetching from a US server has ~200ms.

Connection Pooling

Establishing database connections is expensive (100-300ms). Connection pools maintain a set of open connections that are reused for incoming requests, eliminating connection overhead from the critical path.

Async Processing

If a task doesn't need to complete within the user's request cycle, move it to a background queue. Email sending, report generation, and image processing can all be async — the user gets an immediate response while work happens in the background.

6. Monitoring Performance in Production

You cannot improve what you don't measure. Production monitoring is non-negotiable for serious systems. The key tooling:

- Prometheus — open-source metrics collection and alerting
- Grafana — visualization and dashboard platform
- Datadog — comprehensive monitoring platform (commercial)
- New Relic — application performance monitoring
- AWS CloudWatch / Azure Monitor — cloud-native monitoring

These tools track latency distributions, throughput, error rates, resource utilization, and alert your team when thresholds are breached.

> **Pro Tip:** Set up monitoring before you launch. You cannot diagnose production problems without observability infrastructure in place. This is a professional standard, not a luxury.
>
> **Key Takeaways:** Latency = time per request. Throughput = requests per second. Always measure percentiles (P99) not averages. Reducing latency = caching, query optimization, CDN, connection pooling. Monitor everything in production.

LECTURE 5: Load Balancing — Distributing Traffic at Scale

Load balancing is the traffic controller of distributed systems. It is what enables horizontal scaling to actually work — without it, adding more servers doesn't help because all traffic still flows to the same machine.

1. The Problem Load Balancing Solves

Without a load balancer, you have a simple architecture: all clients connect to a single server. This works at small scale, but every growth milestone reveals new limits. One server can only handle so much CPU, memory, and network. And if that server crashes, your system is completely down.

The load balancer sits between clients and servers. From the client's perspective, there is one endpoint. Behind the scenes, the load balancer distributes requests across a fleet of servers — smoothing traffic, eliminating overload, and routing around failures.

> **Real-World Analogy:** Think of a bank with 10 teller windows. A load balancer is the person at the door who directs customers: 'Window 3 is free. Window 7 is next.' Without them, everyone would rush to window 1.

2. Layer 4 vs. Layer 7 Load Balancers

Load balancers operate at different layers of the networking stack, and the layer determines what information they can use to make routing decisions.

Layer 4 — Network Layer

L4 load balancers make routing decisions based purely on network information: IP addresses and TCP/UDP ports. They don't inspect the content of packets. This makes them extremely fast and efficient — they can handle millions of connections per second.

Use L4 when: speed is critical, you're routing non-HTTP traffic (gaming servers, streaming), or you need very high throughput.

Examples: AWS Network Load Balancer, HAProxy in TCP mode, Linux IPVS

Layer 7 — Application Layer

L7 load balancers operate at the HTTP/HTTPS level. They can read the full request — URL path, headers, cookies, query parameters. This intelligence enables sophisticated routing decisions.

Examples of L7 routing rules:

- Route '/api/*' to API servers, '/static/*' to CDN origin servers
- Route requests with 'X-Beta-User: true' header to the canary release
- Route based on geographic header to the nearest region

- Route WebSocket connections to dedicated WebSocket servers

Examples: NGINX, AWS Application Load Balancer, Envoy Proxy, Traefik

Feature	L4 Load Balancer	L7 Load Balancer
Routing basis	IP address + TCP port	URL, headers, cookies, content
Performance	Extremely fast	Slightly more overhead
Intelligence	Basic	Content-aware
SSL termination	Pass-through only	Can terminate TLS
Use case	High throughput, raw TCP	Web apps, microservices
Examples	AWS NLB, IPVS	NGINX, AWS ALB, Envoy

3. Load Balancing Algorithms

The load balancer must decide which server gets each request. Different algorithms optimize for different goals.

Round Robin

The simplest algorithm. Requests cycle through servers in sequence: Server 1, Server 2, Server 3, Server 1, Server 2, ... Each server gets an equal share of traffic. Best when all servers have identical capacity and requests have similar processing time.

Limitation: ignores actual server load. A server struggling with a slow request gets the same traffic rate as an idle one.

Weighted Round Robin

Like Round Robin but servers have weights. A server with weight 3 receives 3x more requests than a server with weight 1. Use this when servers have different hardware capacities — your beefier servers can be weighted higher.

Least Connections

Route to the server with the fewest active connections. Superior to Round Robin when request processing times vary significantly. If some requests are slow (large file uploads, complex queries), those servers accumulate connections and the load balancer automatically sends new traffic elsewhere.

Least Response Time

Route to the server with the lowest combination of active connections and current response time. This algorithm adapts in real-time to server performance — if a server becomes slow, it naturally receives less traffic.

IP Hash

Hash the client's IP address to determine which server handles the request. The same IP always goes to the same server. This ensures session affinity — if your servers maintain any per-session state, the same client always reaches the same server.

> **Pro Tip:** Prefer stateless server design to avoid the need for IP hash. But when you must use session affinity (legacy systems, WebSocket connections), IP hash is reliable.

4. Health Checks — Routing Around Failures

Load balancers continuously probe the health of backend servers. If a server fails to respond or returns an error code, the load balancer removes it from rotation. Traffic flows only to healthy servers.

Types of Health Checks

- TCP health check — attempt to establish a connection. If it fails, the server is unhealthy
- HTTP health check — send GET /health and expect a 200 OK response
- Custom health check — the /health endpoint performs actual logic: checks DB connectivity, cache, downstream services

When a server passes health checks again after recovery, the load balancer automatically adds it back to rotation. This is called graceful return to service.

5. Global Load Balancing

For global platforms, a single load balancer in one region isn't enough. Users in India shouldn't be routed to a server in the US — the latency would be 200ms+ just for the network hop.

Global load balancing uses DNS-based routing to direct users to the nearest datacenter. When you resolve a global service's domain name, DNS returns the IP of the closest datacenter.

Global Architecture Pattern

- User in Mumbai → DNS → Mumbai datacenter → low latency
- User in London → DNS → Frankfurt datacenter → low latency
- User in New York → DNS → US East datacenter → low latency

If a datacenter goes down, DNS TTLs expire and requests automatically route to the next nearest region. Netflix, Amazon, and Google all use this pattern.

6. Load Balancer as a Single Point of Failure

If your load balancer fails, all traffic fails with it. The solution: run load balancers in high-availability pairs (active-passive or active-active). Cloud load balancers (AWS ALB, Azure Load Balancer) are inherently redundant.

Interview Insight: In system design interviews, whenever you add a load balancer, the interviewer may ask: 'Is the load balancer itself a single point of failure?' Be ready to answer: run multiple load balancers, use cloud-managed load balancers, or use Anycast routing.

Key Takeaways: Load balancers distribute traffic, enable horizontal scaling, and route around failures. L4 is fast; L7 is intelligent. Key algorithms: Round Robin, Least Connections, IP Hash. Health checks automate failure detection. Use global load balancing for multi-region systems.

LECTURE 6: Database Fundamentals — SQL, NoSQL & ACID

Databases are the foundation of nearly every application. Every user action, transaction, and piece of content must be stored somewhere reliably. Understanding database types, their tradeoffs, and the guarantees they provide is essential system design knowledge.

1. SQL — Relational Databases

Relational databases store data in tables with a fixed schema. Tables have columns with defined types, and rows represent individual records. The power of relational databases is in relationships — tables can reference each other via foreign keys, and complex queries can join multiple tables to answer sophisticated questions.

SQL (Structured Query Language) is the universal interface for relational databases. Despite being decades old, SQL remains the dominant data query language for a simple reason: it's expressive, powerful, and well-understood.

Strengths of SQL Databases

- ACID transactions — guaranteed consistency even in failures (explained below)
- Complex queries — joins, aggregations, subqueries, window functions
- Schema enforcement — data integrity at the database level
- Mature ecosystem — decades of tooling, knowledge, and optimization
- Best for: financial systems, e-commerce, healthcare, any domain requiring strict consistency

Popular SQL Databases

- PostgreSQL — open-source, extremely feature-rich, excellent for complex queries
- MySQL — ubiquitous, great performance, widely understood
- SQL Server — Microsoft ecosystem, excellent Azure integration
- SQLite — embedded database, perfect for mobile and local apps
- Amazon Aurora — cloud-native, MySQL/PostgreSQL compatible, high performance

2. NoSQL — Non-Relational Databases

NoSQL databases reject the rigid relational model in favor of flexible data models optimized for specific access patterns. The term 'NoSQL' covers a wide variety of database types — the only thing they have in common is not being relational.

Document Databases

Store data as JSON-like documents. Perfect when your data is hierarchical and naturally belongs together — a user with their addresses, a product with its variants and images. No JOIN operations needed because related data is embedded in the document.

Examples: MongoDB, CouchDB, Amazon DocumentDB

```
{ "userId": 101, "name": "Priya", "orders": [ {"orderId": 1, "items": ["laptop", "mouse"],
"total": 65000} ] }
```

Key-Value Databases

The simplest data model: a key maps to a value. Blazingly fast reads and writes. Perfect for caching, sessions, and simple lookups where you always know the exact key.

Examples: Redis, DynamoDB, Memcached

Column-Family Databases

Store data in column families — groups of related columns. Optimized for analytical queries over massive datasets and write-heavy workloads. Can handle petabytes of data across thousands of nodes.

Examples: Apache Cassandra, HBase

Graph Databases

Model data as nodes and edges — perfect for relationship-heavy data like social networks, recommendation engines, and fraud detection graphs. SQL struggles with recursive relationship queries; graph databases excel at them.

Examples: Neo4j, Amazon Neptune

Time-Series Databases

Optimized for sequences of timestamped data — metrics, IoT sensor data, financial tick data. Efficient compression and fast range queries over time windows.

Examples: InfluxDB, TimescaleDB, Prometheus

3. ACID Properties — The Foundation of Database Reliability

For any system where data correctness matters — banking, healthcare, e-commerce — ACID properties are not optional. ACID is a set of guarantees that ensure database operations are reliable even in the presence of failures.

Atomicity

A transaction is all-or-nothing. Either all operations in the transaction succeed, or none of them do. There is no partial state.

Example: A bank transfer deducts money from Account A and credits Account B. If the credit step fails, the debit must be automatically rolled back. Without atomicity, money disappears.

Consistency

A transaction takes the database from one valid state to another valid state. All integrity constraints, rules, and relationships defined in the schema remain satisfied before and after every transaction.

Example: If a rule says 'account balance cannot be negative,' no transaction can violate this rule. The database rejects any transaction that would leave it in an invalid state.

Isolation

Concurrent transactions execute as if they were run sequentially. One transaction's intermediate state is invisible to other concurrent transactions. This prevents race conditions like two users booking the last seat simultaneously.

SQL databases implement different isolation levels that trade strictness for performance:

Isolation Level	Problems Prevented	Performance
Read Uncommitted	None (allows dirty reads)	Fastest (rarely used)
Read Committed	Dirty reads	Fast (default in PostgreSQL)
Repeatable Read	Dirty and non-repeatable reads	Moderate (default in MySQL)
Serializable	All anomalies	Slowest (highest consistency)

Durability

Once a transaction is committed, it is permanent. Server crashes, power failures, and network outages cannot undo committed transactions. Databases achieve this by writing transactions to persistent storage (disk) and using write-ahead logs.

4. CAP Theorem — The Distributed Database Dilemma

When databases are distributed across multiple nodes (which is required for scale), a fundamental tension emerges. The CAP theorem, proven by Eric Brewer, states that a distributed system can guarantee at most two of three properties simultaneously:

Property	Definition	Real-world meaning
Consistency (C)	All nodes return the same, latest data	Every read reflects the latest write
Availability (A)	Every request gets a response (not an error)	System always responds, even if some nodes fail
Partition Tolerance (P)	System works despite network partitions	Nodes can't communicate but system keeps running

Networks partition (fail) in real distributed systems. So P is not optional — you must tolerate partitions. The real choice is between C and A during a partition:

- CP systems — sacrifice availability for consistency. During a network partition, refuse requests rather than return potentially stale data. Banking systems, financial ledgers.
- AP systems — sacrifice strict consistency for availability. During a partition, keep responding with potentially stale data. Social media feeds, DNS, shopping carts.

Interview Insight: The CAP theorem doesn't mean you choose two and completely ignore the third. Most real systems operate in the gray area, making nuanced tradeoffs based on specific operations. Understanding where your system falls on the spectrum is what matters.

5. How Big Systems Combine Multiple Databases

Production systems at scale rarely use a single database for everything. Different data has different access patterns, consistency requirements, and scale characteristics. A sophisticated architecture might use:

- PostgreSQL — primary transactional data (users, orders, payments) requiring ACID
- Redis — caching layer for hot data, session storage
- Elasticsearch — full-text search, log aggregation
- Cassandra — time-series data, activity feeds, write-heavy workloads
- MongoDB — product catalogs, content management with flexible schemas

This is called polyglot persistence — using the right database for each job. The complexity is real, but so are the performance gains.

Key Takeaways: SQL = structured, ACID, relationships. NoSQL = flexible, scalable, specific access patterns. ACID guarantees atomicity, consistency, isolation, durability. CAP theorem: you can only guarantee 2 of 3 in distributed systems. Use the right database for each job.

LECTURE 7: Database Replication — Scaling Reads & Ensuring High Availability

A single database server is a liability. If it goes down, your entire application loses data access. And as traffic grows, a single server becomes a read throughput bottleneck. Database replication solves both problems by creating synchronized copies of your database on multiple servers.

1. What is Replication?

Database replication is the process of maintaining multiple copies of the same data on different servers. Changes made to one server (the primary) are automatically propagated to the others (replicas). All copies remain synchronized, providing redundancy, read scalability, and disaster recovery.

> **Real-World Analogy:** Think of Google Docs. When you edit a document, those changes sync to everyone viewing it almost instantly. Replication is the same concept applied to databases — one source of truth, synchronized to multiple copies.

2. Why Replication is Essential

Read Scalability

Writes must go to the primary database to maintain consistency, but reads can be distributed across replicas. If you have three replicas, your read capacity is 4x a single server. For read-heavy applications (social media, e-commerce catalogs, news sites), this is transformative.

Example: Instagram has far more reads (viewing photos/profiles) than writes (uploading/posting). Routing reads to replicas means the primary handles only writes — a much lighter load.

High Availability

If the primary database fails, a replica can be promoted to primary. The promotion process takes minutes (or seconds with modern tooling), dramatically reducing downtime compared to recovering from backups. This is called failover.

Geographic Distribution

Replicas can be in different datacenters or geographic regions. Users in India can read from an India replica, users in the US from a US replica. This reduces read latency globally.

3. Primary-Replica Architecture

The most common replication topology. One server is designated the primary (also called master). All writes go to the primary. The primary propagates changes to one or more replicas (also called secondaries or read replicas). Reads can go to any replica.

Operation	Destination	Why
INSERT / UPDATE / DELETE	Primary only	Maintains write consistency
SELECT (general)	Any replica	Distributes read load
SELECT (after write)	Primary or replica	Depends on consistency needs
Transactions	Primary	ACID requires single primary

4. Replication Lag — The Inevitable Tradeoff

Replication is not instantaneous. There is always some delay between when data is written to the primary and when it appears on replicas. This delay is called replication lag.

Replication lag in healthy systems is typically milliseconds. But under load, network issues, or slow replicas, it can grow to seconds — or even minutes in extreme cases.

The Read-After-Write Problem

A user updates their profile name. The write goes to the primary. They immediately navigate to their profile page, which reads from a replica. The replica hasn't received the update yet. The user sees their old name. They're confused and frustrated.

Solutions:

- Read your own writes from the primary — for the user who just wrote, always read from primary
- Sticky routing — after a write, route reads for that user to primary for a short window (1-2 seconds)
- Synchronous replication — wait for replica confirmation before returning success (slower, but eliminates lag)

> **Pro Tip:** For user-visible state changes (profile updates, settings changes), always read from the primary. For feeds, recommendations, and non-critical content, reads from replicas are fine.

5. Synchronous vs. Asynchronous Replication

Type	How it works	Advantage	Disadvantage
Asynchronous	Primary writes, returns success, then replicates	Low write latency	Potential data loss on primary failure
Synchronous	Primary writes, waits for replica ACK, then returns success	No data loss (if replica ACKs)	Slower write latency
Semi-synchronous	Wait for at least one replica to ACK	Balance of speed and safety	Slightly slower than async

Most production systems use asynchronous replication for performance, with the understanding that in a catastrophic primary failure, a small amount of data (seconds of writes) may be lost. For financial systems, semi-synchronous or synchronous replication is used.

6. Multi-Master Replication

In some scenarios, you need multiple databases to accept writes — not just reads. This is multi-master (or active-active) replication. Any node can accept writes, and changes are synchronized across all nodes.

Use cases: globally distributed databases where write performance in each region matters (CockroachDB, Google Spanner, Amazon Aurora Global Databases).

> ⚠ **Warning:** Multi-master replication introduces write conflicts. What happens when two users update the same record on different masters simultaneously? Conflict resolution strategies must be designed carefully. This is genuinely hard to get right.

7. Automatic Failover

When the primary fails, the system must promote a replica to primary without manual intervention. This is automatic failover. The steps:

41. Monitoring system detects primary is unresponsive
42. Cluster management selects the replica with the least replication lag
43. Selected replica is promoted to primary
44. Other replicas are reconfigured to replicate from the new primary
45. Application connection strings are updated (via load balancer or service discovery)
46. System is operational — only downtime is the failover process itself (seconds to minutes)

Tools: Patroni (PostgreSQL), MySQL Router, AWS RDS Multi-AZ, Azure SQL HA.

8. Replication in Cloud Platforms

Every major cloud provider offers managed database replication as a service:

- AWS RDS Read Replicas — point-and-click replica creation, automatic failover with Multi-AZ
- AWS Aurora — storage-level replication to 6 copies across 3 availability zones, 15 read replicas
- Azure SQL Database — active geo-replication, auto-failover groups
- Google Cloud SQL — read replicas, automated failover

For most applications, managed cloud replication is significantly more practical than setting up your own. The operational complexity of managing replication is substantial.

> ⚠ **Interview Insight:** In a system design interview, mention replication as your first strategy for database high availability. Then clarify: are you using primary-replica for reads, or multi-master for globally distributed writes? Show that you understand the tradeoffs.

Key Takeaways: Replication = synchronized copies of your database. Primary handles writes; replicas handle reads. Replication lag is real — design around it. Automatic failover enables high availability. Cloud platforms make replication straightforward. Replication scales reads; sharding (Lecture 8) scales writes.

LECTURE 8: Sharding & Partitioning Strategies

Replication creates copies of your data, which helps with reads and availability. But what happens when your dataset grows so large that a single server can't hold it — not even with more RAM and a bigger disk? That's when sharding enters the picture.

1. The Problem That Demands Sharding

Your application has been growing for years. The users table has 2 billion rows. A simple SELECT with a WHERE clause that used to take 5ms now takes 3 seconds. The primary database CPU runs at 95% just from reads. Even after adding replicas, the primary is overwhelmed by write load.

Vertical scaling won't help — you're already on the largest available machine. Replication won't help — all those replicas get the same queries, and writes still only go to the primary. You need to split the data.

> ⍰ **Real-World Analogy:** Imagine a library with 10 million books crammed into a single room. Finding any book is slow. Librarians are overwhelmed. The solution: build 4 smaller libraries, each holding 2.5 million books organized by subject. Now queries are 4x faster, capacity is 4x larger, and each library can be managed independently.

2. What is Sharding?

Sharding (also called horizontal partitioning) means splitting a large database into smaller, faster, more manageable pieces called shards. Each shard holds a subset of the total data and runs on a separate server. Collectively, the shards contain all the data.

A shard router (also called a query router or shard coordinator) sits between the application and the shards. The application sends queries to the router, which determines which shard(s) contain the relevant data and forwards the query accordingly.

3. Sharding Strategies

Range-Based Sharding

Data is split into ranges based on a shard key value. Shard 1 holds user IDs 1–1,000,000, Shard 2 holds 1,000,001–2,000,000, etc.

Advantage: Simple to reason about. Range queries ('give me all orders from July') are efficient — they touch only relevant shards.

Disadvantage: Hotspots. If new users are always assigned the highest IDs, the highest-range shard gets all the traffic while older shards sit idle. This uneven load distribution is called a 'hot shard'.

Hash-Based Sharding

Apply a hash function to the shard key, then use modulo to determine the shard: shard = hash(userId) % numberOfShards

This distributes data evenly across shards regardless of the key distribution. Hash-based sharding prevents hot shards.

Disadvantage: Range queries become expensive — data for a range of user IDs is spread across all shards, requiring a scatter-gather query that hits every shard.

Another critical disadvantage: adding or removing shards requires rehashing all data. With 4 shards, user 101 goes to shard 1. Add a 5th shard and the modulo changes — user 101 now hashes to shard 3, but data is still in shard 1.

> **Pro Tip:** Consistent hashing solves the resharding problem. Instead of modulo, it uses a ring structure where adding a shard only moves a fraction of the data. Used by Cassandra, Amazon DynamoDB, and distributed caches.

Directory-Based Sharding

A lookup service (directory) maintains a mapping from shard key to shard location. The application queries the directory to find where a record lives.

Advantages: Extremely flexible. You can move data between shards without changing the application. Non-uniform shard sizing is possible.

Disadvantages: The directory itself is a single point of failure and must be replicated. Every query requires an extra lookup — latency increases.

Geographic Sharding

Data is partitioned by geographic location. Indian user data lives in an India datacenter; US user data in a US datacenter. This provides data locality (reducing cross-region latency), helps with data residency compliance (GDPR requires EU data to stay in the EU), and improves read performance.

4. Choosing the Right Shard Key

The shard key is the field used to determine which shard a record belongs to. Choosing the right shard key is perhaps the most consequential design decision in a sharded system. A bad shard key leads to:

- Hot shards — uneven distribution means some shards are overwhelmed
- Cross-shard queries — operations that span multiple shards are slow and complex
- Rebalancing nightmares — data must be moved when shards become unbalanced

Properties of a good shard key:

- High cardinality — many possible values, enabling fine-grained distribution
- Even distribution — values spread evenly across the range (avoid timestamps as shard keys — all new data goes to the latest shard)
- Query locality — most queries should touch only one shard

5. Challenges of Sharding

Cross-Shard Queries

SELECT COUNT(*) FROM users runs on a single database in milliseconds. In a sharded system, this query must run on all 10 shards, collect results, and aggregate. This 'scatter-gather' pattern is 10x more expensive and slower.

Design to avoid cross-shard queries as much as possible. When they're unavoidable, accept the latency cost or pre-compute aggregates asynchronously.

Cross-Shard Transactions

Transactions spanning multiple shards require distributed transaction coordination (two-phase commit or sagas), which is complex and slow. Avoid designing data models that require transactions across shards.

Rebalancing

As data grows, shards become unequal in size. You'll need to split hot shards and move data to new shards. This rebalancing process must happen without downtime — extremely challenging at scale.

Joins Across Shards

If Users are in Shard 1 and Orders are in Shard 2, a JOIN query becomes a complex distributed operation. The typical solution: denormalize data by duplicating the relevant fields across shards, eliminating the need for joins.

6. Real-World Sharding Examples

Company	System	Shard Key	Strategy
Facebook	User data	User ID	Hash-based with consistent hashing
Instagram	Photos, Likes	User ID	ID-based sharding
Twitter	Tweets	Tweet ID (time-based)	Range-based
Amazon	Orders	Customer ID + Region	Geo + hash combination
Uber	Trip data	City ID	Geographic sharding

Interview Insight: Sharding is complex. Don't shard prematurely. Start with vertical scaling, then replication, then caching. Shard only when those strategies have been exhausted and you have data too large or write throughput too high for a single server. Most applications never need to shard.

Key Takeaways: Sharding splits data across multiple database servers. Four strategies: range, hash, directory, geographic. Choose shard key carefully — it determines distribution and query efficiency. Cross-shard queries, transactions, and joins are expensive. Shard last — it adds significant complexity.

LECTURE 9: Caching Fundamentals — The Most Powerful Optimization

Caching is the practice of storing frequently accessed data in a faster storage layer so future requests can be served without touching slower storage. It is the single most impactful performance optimization available to most systems, reducing response times by 10-100x and dramatically reducing database load.

1. Why Caching is Necessary

Hardware speed differences are enormous. RAM is ~100x faster than SSD for read operations. Serving data from memory instead of a database query can reduce response time from 50ms to under 1ms.

Storage Layer	Typical Read Speed	Typical Write Speed
CPU L1 Cache	~0.5 ns	~0.5 ns
CPU L3 Cache	~10 ns	~10 ns
RAM (Redis cache)	~100 ns – 1 µs	~100 ns – 1 µs
SSD (Local disk)	~50–150 µs	~50–150 µs
Database (network + disk)	~1–50 ms	~1–50 ms
Remote API call	~50–500 ms	~50–500 ms

When 100,000 users request the same product page simultaneously, without caching you'd execute 100,000 database queries. With caching, you execute 1 query — the result is cached and served to everyone else from memory.

2. Cache Hit and Cache Miss

Every cache interaction results in one of two outcomes:

- Cache Hit — the requested data is found in the cache. Return it immediately. Fast path. No database involved.
- Cache Miss — the data is not in cache. Fetch from the database, store in cache, return to client. Slow path, but subsequent requests hit the cache.

Cache hit ratio = (cache hits) / (total requests). A cache hit ratio above 90% is good; above 95% is excellent. Below 80% suggests cache misses are frequent enough to reconsider what you're caching and your eviction strategy.

3. Cache Eviction Policies

Caches have finite memory. When the cache is full, something must be removed to make room for new data. The eviction policy determines what gets removed.

Policy	How it works	Best for
LRU (Least Recently Used)	Remove data not accessed for the longest time	General purpose — most popular choice
LFU (Least Frequently Used)	Remove data accessed least often	When some data is permanently more popular
FIFO (First In, First Out)	Remove data added longest ago	Simple scenarios, rarely optimal
TTL (Time To Live)	Remove data after a configured time	Data with natural expiration (session tokens)
Random	Remove random entries	When no access pattern information is available

Redis uses LRU by default and allows configuration of alternative policies per dataset.

4. Caching Strategies — Where and When to Cache

Cache-Aside (Lazy Loading) — Most Common

The application manages the cache manually. On every read: check cache first. If hit, return. If miss, fetch from database, populate cache, return. The cache is populated lazily — only when data is first requested.

Advantages: Simple to implement. Only actually requested data gets cached — no wasted memory on data nobody needs. Cache failures are transparent — the database is always the fallback.

Disadvantages: Cache miss = full latency on first request. Data can go stale if database is updated without invalidating cache.

> **Pro Tip:** Cache-aside is the right default for most applications. Start here.

Write-Through

Every database write also writes to the cache. The cache is always synchronized with the database. No stale data.

Advantages: Reads are always fast (data is in cache before it's needed). No stale data problem.

Disadvantages: Write path is slower (must write to both). Data that's written but never read wastes cache memory.

Write-Behind (Write-Back)

Writes go to the cache first; the cache asynchronously writes to the database later. Extremely low write latency — the application doesn't wait for database acknowledgment.

Advantages: Ultra-fast writes. Very low database write pressure.

Disadvantages: Risk of data loss if the cache fails before flushing to database. Significantly more complex to implement reliably.

Read-Through

The cache sits between the application and database. The application only talks to the cache. On a miss, the cache fetches from the database automatically. The application code is simplified — it never talks to the database directly.

5. Cache Invalidation — The Hard Problem

Phil Karlton famously said: 'There are only two hard things in Computer Science: cache invalidation and naming things.'

Why is invalidation hard? Because stale data in cache causes correctness bugs. If a user updates their email but the cache still has the old value, the application shows incorrect data. But if you invalidate too aggressively, you lose the performance benefit of caching.

Strategies for cache invalidation:

- TTL-based expiration — data expires after a fixed time. Simple but causes occasional stale reads.
- Event-based invalidation — when the underlying data changes, explicitly delete/update the cache entry. Precise but requires coordination between write and cache code.
- Cache-aside with short TTL — invalidation happens naturally at TTL expiry. Small stale window acceptable for most use cases.
- Write-through — cache is always updated on write, so it never goes stale.

> ⚠ **Warning:** Never cache data that must always be fresh — bank account balances, inventory levels for the last few items, medical records. The cost of showing stale data outweighs the performance benefit.

6. What to Cache and What Not To

Good to Cache	Avoid Caching
Product catalogs and descriptions	Bank account balances
User profiles (acceptable staleness)	Payment confirmation state
Homepage content	Real-time inventory for last item
Configuration and feature flags	Sensitive personal data (GDPR risk)
Computed aggregates (top 10 lists)	Passwords and authentication tokens
Static reference data (country codes)	Data that changes faster than your TTL

7. Advanced Cache Problems

Cache Stampede (Thundering Herd)

Popular cache entry expires. Simultaneously, thousands of requests miss and all hit the database. Database is overwhelmed. Cache stampede.

Solutions: Use mutex locking — only one request fetches the data, others wait. Or use probabilistic early expiration — start refreshing cache entries slightly before they expire, eliminating the stampede window.

Cache Penetration

Requests for data that doesn't exist in the database. Since there's nothing to cache, every request hits the database. Attackers can exploit this deliberately with invalid IDs.

Solution: Cache null values with a short TTL. If the database returns empty, cache 'EMPTY' for 1 minute. Subsequent requests hit the cache, not the database.

Cache Avalanche

Many cache entries expire simultaneously (e.g., at midnight when TTLs are set to 24 hours). Massive wave of cache misses. Database overloaded.

Solution: Jitter — add a random offset to TTLs so expiration is spread out. Instead of TTL=86400 for everything, use TTL = 86400 + random(0, 3600).

> **Key Takeaways:** Caching serves frequently read data from fast memory instead of slow disk/database. Hit ratio > 90% is the target. Cache-aside is the standard pattern. Invalidation is the hardest problem. Never cache data that must always be fresh. Watch for stampede, penetration, and avalanche patterns.

LECTURE 10: Storage Systems — File, Blob & Object Storage

Not all data is structured rows and columns. Modern applications deal with files — PDFs, images, videos, audio, and binary data. Storing these incorrectly (like in a database BLOB column) leads to performance disasters at scale. This lecture covers the three fundamental types of storage systems and when to use each.

1. The Three Storage Types

Type	Structure	Access Method	Scalability	Best Use Case
File Storage	Folders and files	File system path	Low–Medium	Local systems, small apps
Blob Storage	Containers + blobs	API or URL	Medium–High	Large binary files
Object Storage	Buckets + objects (with rich metadata)	HTTP URL (REST API)	Virtually unlimited	Cloud-native apps, global scale

2. Object Storage — The Modern Standard

Object storage is the dominant paradigm for large-scale file storage in cloud-native systems. Each file is stored as an 'object' with three components: the data itself, metadata (key-value pairs describing the object), and a globally unique identifier.

Access is purely via HTTP REST APIs — no file system paths, no mounts, no drivers. This makes object storage globally accessible, naturally distributed, and infinitely scalable.

Key Object Storage Services

- AWS S3 (Simple Storage Service) — industry standard, 99.999999999% (11 nines) durability
- Azure Blob Storage — Microsoft's equivalent, deep Azure ecosystem integration
- Google Cloud Storage — Google's equivalent, strong ML/AI integration
- MinIO — open-source S3-compatible storage for on-premises

> ✓ **Pro Tip:** The standard pattern: store files in object storage (S3), store only the URL in your database. Never store file contents directly in the database.

3. The Correct Architecture for File Handling

47. User uploads file to your API
48. API server streams file directly to object storage (S3, Azure Blob)
49. Object storage returns the file's URL

50. API stores that URL in the database (along with metadata: filename, size, uploader, etc.)
51. When displaying the file, render the URL — user's browser fetches directly from object storage or CDN

Why stream directly to object storage? The API server should not buffer large files in memory. At scale, buffering every upload in your API server RAM would exhaust memory quickly.

4. Storage Durability and Availability

When choosing a storage system, understand these guarantees:

- Durability: Will data survive hardware failures? AWS S3 promises 99.999999999% durability — if you store 10 million files, you can expect to lose less than one file per 100,000 years.
- Availability: Can you access data when you need it? S3 Standard offers 99.99% availability.
- Redundancy: Object storage achieves high durability by automatically replicating data across multiple availability zones within a region.

> ⍰ **Interview Insight:** Interview question: 'Where will you store user uploaded images?' Perfect answer: 'Object storage (S3), with CDN in front for global performance. Database only stores the object URL and metadata.'

LECTURE 11: Consistency Models — Strong vs. Eventual Consistency

In a single-server system, consistency is automatic — there's only one copy of the data, so everyone sees the same thing. In distributed systems with multiple nodes and replicas, maintaining a consistent view of data is a fundamental challenge. This lecture explores the consistency spectrum.

1. The Core Question

After a write, when will all readers see the updated value? The answer determines your consistency model.

> 🤔 **Real-World Analogy:** Imagine editing a shared Google Doc. With strong consistency, as soon as you type a letter, every collaborator sees it immediately. With eventual consistency, your letter appears to your collaborators after a brief delay — but eventually everyone agrees on the current state.

2. Strong Consistency

After a successful write, all subsequent reads from any node return the updated value. No reader ever sees stale data. The system behaves as if there is only one copy of the data.

How Strong Consistency is Achieved

The primary (master) must ensure all replicas have applied the write before returning success to the client. This requires synchronous replication — the write isn't acknowledged until all (or a quorum of) replicas confirm receipt.

Cost of Strong Consistency

- Higher write latency — must wait for all replicas to acknowledge
- Reduced availability — if replicas are unreachable (network partition), writes may fail
- Limited scalability — coordinating across many nodes is expensive

Use Strong Consistency When

- Financial transactions — wrong balance shown = severe problem
- Inventory management — showing 1 item available when it's already sold
- Authentication — showing a user as logged in when they've logged out

3. Eventual Consistency

After a write, different nodes may temporarily see different values. But given enough time with no new writes, all nodes will eventually converge to the same value. The window of inconsistency is typically milliseconds to seconds.

Why Eventual Consistency is Often Acceptable

For most social media interactions, it doesn't matter if a like count shows 10,042 for 2 seconds and then updates to 10,043. The system is functionally correct — just briefly stale.

Use Eventual Consistency When

- Social media feeds — 2 second lag on a new post is imperceptible
- Product recommendations — slightly stale suggestions are fine
- View counts, like counts — approximate values are acceptable
- Search indexes — can lag slightly behind the source database

4. Consistency Models Spectrum

Model	Guarantee	Performance	Example Systems
Linearizability (Strong)	Reads always see latest write; global order	Slowest	Zookeeper, etcd, Google Spanner
Sequential Consistency	All nodes see same order, not necessarily latest	Slow	Some distributed databases
Causal Consistency	Causally related ops appear in order	Moderate	Some graph DBs, MongoDB
Eventual Consistency	Will converge; temporary inconsistency allowed	Fast	Cassandra, DynamoDB, DNS
Read-Your-Writes	You always see your own writes immediately	Moderate	Session-pinned reads

Interview Insight: In interviews, when asked about consistency, don't just say 'strong' or 'eventual'. Show nuance: 'For payment processing, we use strong consistency. For the activity feed, eventual consistency is acceptable because a few seconds of stale data has no business impact.'

LECTURE 12: Distributed Systems Fundamentals

A distributed system is a collection of independent computers that work together to appear as a single system to users. This is the architectural reality of modern software at scale — not one big server, but many small ones collaborating.

1. Why Distributed Systems Exist

Single servers hit physical limits: maximum CPU cores, maximum RAM, maximum network bandwidth. Beyond a certain scale, no single machine can handle the load. The only option is to distribute the work across many machines.

Additionally, single servers are single points of failure. Distributed systems can survive individual machine failures — other nodes take over. This resilience is fundamental to high-availability systems.

2. The Eight Fallacies of Distributed Computing

L Peter Deutsch at Sun Microsystems identified 8 assumptions that engineers incorrectly make about distributed systems. Understanding these fallacies prevents catastrophic design mistakes:

52. The network is reliable — it is not. Packets drop, routers fail, cables get cut.
53. Latency is zero — every network call has latency. Design accordingly.
54. Bandwidth is infinite — you can saturate network links. Be conscious of data transfer.
55. The network is secure — assume adversarial networks. Encrypt everything.
56. Topology doesn't change — servers come and go. Use service discovery.
57. There is one administrator — different parts of your system may have different owners.
58. Transport cost is zero — network calls have CPU, bandwidth, and dollar cost.
59. The network is homogeneous — different services may use different protocols and formats.

3. Core Challenges

Partial Failures

In a single process, if something fails, everything fails. In a distributed system, some parts fail while others continue. This is actually good for resilience, but it creates a new problem: the system must gracefully handle partial functionality.

A payment service calling an email notification service should not fail the payment if the email service is down. The email is a non-critical side effect. The payment must succeed.

Network Unreliability

Network calls can fail in ways that local function calls cannot: timeouts, packet loss, partial sends, connection resets. Unlike a local function call that either succeeds or throws an exception, a remote call can fail with an 'unknown' outcome — the server may have processed it, or may not have.

Clock Synchronization

Each machine has its own clock, and clocks drift. Two machines may disagree about the current time by milliseconds. This makes ordering events across machines non-trivial. Do not assume that timestamps from different machines are comparable. Use vector clocks or logical timestamps for ordering distributed events.

Data Consistency

When data is stored across multiple nodes, keeping them synchronized is an ongoing challenge. The CAP theorem (Lecture 6) provides the theoretical framework; practical distributed databases implement specific consistency models to manage this.

4. The Design Mindset Shift

In a single-process system, you think in function calls: 'Call this function, get this result.' In a distributed system, you think in network requests: 'Send this message, and the reply might come, might not, might come multiple times, might come slowly.'

This shift requires designing for:

- Retries — because requests fail transiently
- Idempotency — because retried requests might succeed multiple times
- Timeouts — because waiting forever is unacceptable
- Circuit breakers — because a slow dependency shouldn't cascade failures
- Compensation — because distributed transactions need explicit rollback logic

> ⍰ **Interview Insight:** Failure in distributed systems is not exceptional — it is routine. Your job is not to prevent failures but to contain them, recover from them, and ensure the system remains functionally correct despite them.

LECTURE 13: Data Modeling at Scale

Data modeling is the art and science of structuring your data so that your most common operations are fast and your rarest operations are acceptably slow. At small scale, data model decisions are forgiving. At scale, a bad data model is an existential threat to your system's performance.

1. The Fundamental Rule: Query First, Then Design

The most common mistake in data modeling is designing tables first and then writing queries. At scale, this approach fails. The correct sequence:

60. Understand your access patterns — what queries will run most frequently?
61. Identify your read-to-write ratio — is this read-heavy or write-heavy?
62. Choose your data model to optimize for your most common operations
63. Accept tradeoffs for rare operations

> ✓ **Pro Tip:** Ask yourself: What does my system do 100,000 times per second? Design the data model to make that operation as fast as possible.

2. Normalization vs. Denormalization

Normalization — The SQL Ideal

Normalization eliminates data duplication by storing each fact in exactly one place. A user's name is stored once in the Users table. If they change their name, one update propagates everywhere.

The cost: reads require JOINs across tables. In distributed systems, JOINs across shards are extremely expensive. Normalization is a design choice optimized for write correctness at the expense of read performance.

Denormalization — The Scale Reality

Denormalization deliberately duplicates data to eliminate JOINs. Store the user's name in every order record. Now fetching an order doesn't require joining Users table — it's all in one row.

The cost: data becomes stale when the source changes. Updating a user's name means updating every order record. But reads are fast.

At scale, systems overwhelmingly choose denormalization because reads vastly outnumber writes (typically 100:1 or higher). Optimizing the common case (reads) at the expense of the rare case (updates) is rational.

3. Indexing — The Superpower

An index is a data structure that enables fast lookup of records by a specific column. Without an index, a database must scan every row (full table scan) to find matching records. With an index, it jumps directly to matching rows.

Types of indexes:

- B-Tree index — standard for range queries and equality lookups. Default index type in PostgreSQL and MySQL.
- Hash index — ultra-fast for equality lookups, useless for ranges.
- Composite index — indexes on multiple columns. Order matters — index on (city, age) helps queries filtering by city but not by age alone.
- Partial index — index covering only rows matching a condition. Smaller, faster.
- Full-text index — enables text search with relevance ranking.

> ⚠ **Warning:** Indexes aren't free. Each index adds write overhead (the index must be updated on every insert/update/delete) and uses disk space. Don't over-index. Focus indexes on columns used in WHERE clauses and JOIN conditions of frequent queries.

LECTURE 14: Pagination & Handling Large Datasets

When your API returns data from a table with millions of rows, returning all of it in one response is not an option. The response would be gigabytes large, the server would run out of memory, the network would be saturated, and the client would crash. Pagination is the solution — breaking large datasets into manageable chunks.

1. Offset-Based Pagination

The simplest approach. Specify a page number and page size. The database skips (OFFSET) the first N rows and returns the next N.

```
SELECT * FROM orders ORDER BY created_at DESC LIMIT 20 OFFSET 100;
```

Client calls: GET /orders?page=6&size=20 (skip 100, return rows 101-120)

Advantages: Simple to implement, easy to understand, supports jumping to any page.

Critical disadvantage: Performance degrades severely at high offsets. OFFSET 1000000 makes the database scan and discard 1 million rows before returning 20. At large scale, offset pagination becomes unusably slow.

2. Cursor-Based Pagination — The Production Standard

Instead of a page number, use a cursor — a pointer to a specific position in the dataset, typically an encoded representation of the last item returned.

```
SELECT * FROM orders WHERE id > last_id ORDER BY id LIMIT 20;
```

Client calls: GET /orders?cursor=eyJpZCI6MTAwfQ&size=20

The cursor encodes the ID (or timestamp) of the last item seen. Each page starts from exactly where the last one ended, using an indexed query — no offset, no scan.

Advantages: Consistent performance regardless of position. Handles real-time data changes gracefully (inserted/deleted rows don't cause duplicate or missing items). Natural fit for infinite scroll.

Disadvantages: Cannot jump to arbitrary pages. Cannot tell total count without a separate query.

> **Pro Tip:** Instagram, Twitter, Facebook, and LinkedIn all use cursor-based pagination for their feeds. If you're building a feed or timeline, cursor pagination is the right choice.

3. Keyset Pagination — Maximum Performance

Similar to cursor-based but uses actual column values rather than encoded pointers. The WHERE clause uses the value of the last record's sort key.

```
SELECT * FROM orders WHERE (created_at, id) < ('2026-01-15', 12345) ORDER BY created_at
DESC, id DESC LIMIT 20;
```

By using a composite key that matches the index, keyset pagination achieves maximum performance — the database goes directly to the right position via the index without scanning anything.

Pagination Type	Performance	Arbitrary Pages	Real-time Safe	Use Case
Offset	Degrades at high offsets	Yes	No	Admin panels, small datasets
Cursor	Consistent, fast	No	Yes	Feeds, timelines, mobile apps
Keyset	Maximum performance	No	Yes	High-scale APIs

LECTURE 15: Rate Limiting — Protecting Systems from Overload

Rate limiting is the practice of restricting how many requests a client can make within a time window. Without rate limiting, a single misbehaving client can exhaust your system's resources — accidentally (a buggy retry loop) or deliberately (a DDoS attack).

1. Why Rate Limiting Matters

A public API without rate limiting is an open invitation to abuse. A client with a bug could send 10,000 requests per second, overwhelming your server and blocking legitimate users. A malicious actor could deliberately hammer your auth endpoints to brute-force passwords.

Rate limiting provides:

- Resource protection — prevent any single client from monopolizing server resources
- DDoS mitigation — limit the effectiveness of denial-of-service attacks
- Cost control — API calls have infrastructure cost; limits prevent runaway bills
- Fair usage — ensure all clients get a reasonable share of capacity

2. Rate Limiting Algorithms

Fixed Window

Counts requests in a fixed time window. Window resets at a fixed interval (e.g., every minute on the minute). Simple to implement.

Problem: Clients can double their effective rate by sending requests at the end of one window and immediately at the start of the next.

Sliding Window

Tracks requests over a rolling window — always the last N seconds, not a fixed interval. More accurate than fixed window, prevents the boundary burst problem.

Implementation: Store timestamps of recent requests. Count how many fall within the window. More memory-intensive than fixed window.

Token Bucket — Industry Standard

A bucket holds tokens. Tokens refill at a constant rate. Each request consumes one token. If the bucket is empty, the request is rejected.

This elegantly handles bursts: if a client hasn't made requests for a while, they accumulate tokens and can burst. But sustained high rates drain the bucket.

Used by: AWS API Gateway, Stripe, GitHub, most major APIs.

Leaky Bucket

Requests are processed at a constant rate, regardless of incoming rate. Burst requests are queued; if the queue is full, requests are dropped. Provides a smooth, constant output rate — useful for traffic shaping.

3. Rate Limit Dimensions

Rate limits can be applied at different granularities:

- Per IP address — limits anonymous clients
- Per API key — limits authenticated clients
- Per user ID — limits authenticated users
- Per endpoint — stricter limits for expensive operations (password reset, bulk export)
- Per tenant — limits in multi-tenant SaaS systems

4. Rate Limit Response Headers

Well-designed APIs communicate rate limit status to clients via headers:

```
X-RateLimit-Limit: 1000         # Total requests allowed per window
X-RateLimit-Remaining: 750      # Requests remaining in current window
X-RateLimit-Reset: 1703123456   # Unix timestamp when window resets
Retry-After: 30                 # Seconds until client can retry (on 429)
```

Return HTTP 429 Too Many Requests when rate limit is exceeded.

> **Interview Insight:** In system design interviews, when designing a public API, always include rate limiting. Describe the algorithm (token bucket is usually the right answer), the limit dimensions (per API key), and how you'd implement it (Redis with atomic increment operations).

LECTURE 16: Fault Tolerance — Building Systems That Survive Failures

Fault tolerance is not about preventing failures — at scale, failures are guaranteed. It's about ensuring that when components fail (and they will), the system continues to function correctly, degrades gracefully, and recovers automatically.

1. Circuit Breaker Pattern

Named after electrical circuit breakers, this pattern prevents cascading failures when a dependency is failing. Instead of letting your service wait endlessly for a broken downstream service, the circuit breaker detects the failure and short-circuits further calls.

Three States

- Closed — normal operation. Requests flow through. Failures are counted.
- Open — failure threshold exceeded. All requests immediately fail without attempting the call. No resources wasted waiting. After a cooldown period, transitions to Half-Open.
- Half-Open — allow limited test requests through. If they succeed, close the circuit. If they fail, open again.

The circuit breaker provides 'fail fast' behavior. Instead of timing out after 30 seconds for every failed call, requests fail in milliseconds — freeing threads to handle other work.

> **Real-World Analogy:** Calling a restaurant. They're not picking up. After 3 failed attempts, you stop calling for 10 minutes (circuit open). Then you try once more to see if they're open (half-open). If they answer, you resume normal calls (circuit closed).

2. Timeout Configuration

Never wait forever for a response from a dependency. Configure timeouts for all external calls: database queries, HTTP requests to downstream services, cache operations, message queue operations.

Without timeouts: a slow database query holds a thread for minutes. With 1000 concurrent users, you quickly exhaust all threads, and new requests queue up, then timeout. The entire server becomes unresponsive — not because of load, but because of one slow database.

Timeout Hierarchy

- Connection timeout — time to establish the connection (typically 1-5 seconds)
- Read timeout — time waiting for data after connection (typically 5-30 seconds, lower for user-facing APIs)
- Overall request timeout — total time budget for the entire operation

> **Pro Tip:** Set timeouts based on your SLA. If users expect a 200ms response, your downstream service timeout should be 100ms — leaving the remaining 100ms for your own processing.

3. Retry Strategies

Many failures are transient — network glitches, momentary overload, brief unavailability. Retrying these requests often succeeds.

Retry Anti-Patterns

Immediate retry loop with no delay is dangerous. If a server is overwhelmed, hammering it with retries makes the problem worse, potentially crashing it completely.

Exponential Backoff with Jitter

Retry with increasing delays: 100ms → 200ms → 400ms → 800ms. This gives the failing service time to recover. Adding random jitter (randomness to the delay) prevents 1000 clients all retrying at the same moment.

```
Retry 1: wait 100ms + random(0, 50ms)
Retry 2: wait 200ms + random(0, 100ms)
Retry 3: wait 400ms + random(0, 200ms)
```

4. Bulkhead Isolation

In ship design, bulkheads are watertight compartments. If one compartment floods, the others stay dry. Software bulkheads isolate components so one failure doesn't cascade.

Implementation: Separate thread pools for different types of work. If report generation saturates its thread pool, login and payment processing still have their own pools and continue working.

5. Graceful Degradation

When a non-critical component fails, the system should continue serving its core functionality with reduced features rather than failing completely.

Netflix example: If the recommendation system fails, the homepage still loads. Users see a generic list instead of personalized recommendations. Core functionality (streaming) is unaffected.

> ⍰ **Interview Insight:** Design your system as a collection of tiers: core functionality (must work), enhanced features (should work), and nice-to-haves (can fail). Make each tier independently deployable and independently degradable.

> ⍰ **Key Takeaways:** Fault tolerance = containing and recovering from failures. Circuit breakers prevent cascading failures. Timeouts prevent resource exhaustion. Retries with exponential backoff handle transient failures. Bulkheads isolate components. Graceful degradation maintains core functionality when non-critical components fail.

LECTURE 17: Retries, Idempotency & Safe APIs

Retries solve transient failures. But naive retries can cause duplicate operations — charging a user twice, creating two orders, sending multiple emails. Idempotency is the design property that makes retries safe.

1. The Scary Reality of Distributed Failures

Consider this scenario: A user clicks 'Pay Now'. Your server processes the payment successfully, then tries to send the response. The network drops. The client never receives the success response. The client assumes failure and retries the payment. Your server processes the payment again. The user is charged twice.

This is not an edge case. Network failures happen constantly in production systems. The solution is idempotency.

2. Idempotency Defined

An operation is idempotent if performing it multiple times has the same effect as performing it once. Mathematically: f(f(x)) = f(x).

Operation	Idempotent?	Why
GET /user/101	Yes	Fetching doesn't change state
DELETE /order/5	Yes	Deleting already-deleted resource = same result
PUT /user/101 {name: 'Priya'}	Yes	Setting to a value is idempotent
POST /payment {amount: 1000}	No (by default)	Creates a new payment each time
PATCH /account/1 {balance: +500}	No	Relative changes are not idempotent

3. The Idempotency Key Pattern

For non-idempotent operations that need to be retried safely (like payments), clients include an idempotency key — a unique identifier for this specific logical operation.

```
POST /payment
Idempotency-Key: abc-def-ghi-123
{ "amount": 1000, "currency": "INR" }
```

Server behavior:

64. Extract the Idempotency-Key from the request header
65. Check storage (Redis or database) — has this key been seen before?

66. If not seen: process the payment, store (key → response) with TTL
67. Return response to client
68. If seen again (retry): look up stored response, return it immediately without processing

> ✓ **Pro Tip:** Use Redis with TTL for idempotency key storage. Set TTL to 24 hours for payments, shorter for less critical operations. The client must generate the key before sending the request — typically a UUID.

4. Designing Safe APIs

Best practices for APIs that handle retries gracefully:

- Use PUT and DELETE over POST for operations that can be retried — HTTP semantics define them as idempotent
- Accept and process idempotency keys for all payment and state-changing operations
- Return the same response for duplicate requests — don't fail with 'already processed'
- Log all operations with their idempotency keys for audit trails
- Design operations as state transitions rather than incremental changes

LECTURE 18: Caching Deep Dive — Redis, CDN & Advanced Strategies

Lecture 9 introduced caching concepts. This lecture goes deeper — into Redis internals, distributed caching architecture, CDN design, and advanced cache problems you'll face in production.

1. Redis — The Swiss Army Knife of Caching

Redis (Remote Dictionary Server) is an in-memory data structure store. It's not just a key-value cache — it supports rich data structures that enable sophisticated caching patterns.

Data Structure	Operations	Cache Use Case
Strings	GET, SET, INCR, EXPIRE	Simple key-value caching, counters
Hashes	HGET, HSET, HMGET	Cache user profiles (partial updates)
Lists	LPUSH, RPOP, LRANGE	Recent activity feeds, queues
Sets	SADD, SMEMBERS, SINTERSTORE	User permissions, unique visitors
Sorted Sets	ZADD, ZRANGE, ZRANGEBYSCORE	Leaderboards, rate limiting
Streams	XADD, XREAD	Event streaming, pub/sub

Redis in Distributed Architectures

Unlike in-process caches (where each server has its own cache), Redis is a separate process that all application servers share. This means:

- All servers see the same cache state — no cache inconsistency between servers
- Cache survives application server restarts
- Cache can be scaled independently from application servers
- Redis Cluster supports sharding for datasets larger than one server's RAM

2. Cache Warming Strategies

A cold cache (after deployment or failure) causes a thundering herd — all requests miss and hit the database simultaneously. Cache warming addresses this.

Pre-warming

Before deploying a new version or restarting, proactively populate the cache with likely-to-be-requested data. Run a background job that fetches the top-N products, most active users, and common queries, and loads them into cache.

Gradual Traffic Shift

When deploying behind a load balancer, gradually shift traffic to new instances while old instances are still warm. By the time new instances handle full traffic, they've had time to warm their caches.

3. Cache Topology Patterns

Look-Aside Cache (Cache-Aside)

The application checks cache, falls back to database on miss, populates cache. Standard pattern — already covered in Lecture 9.

Sidecar Cache

Each application instance has a local cache (high-speed, small size) in addition to the shared distributed cache. Hot data is in local memory; warm data is in Redis; cold data is in the database. Three-tier cache hierarchy.

Read-Through / Write-Through Cache

The cache layer handles database interaction transparently. Application only talks to the cache; the cache manages database reads and writes. Reduces application complexity but adds cache layer dependency.

4. Production Cache Monitoring

Key metrics to monitor in production:

- Hit rate — should be above 90%; declining hit rate indicates configuration issues
- Memory utilization — Redis OOM (out of memory) is catastrophic; set maxmemory and eviction policy
- Latency — p99 cache latency should be < 2ms
- Key eviction rate — high eviction rate means cache is undersized
- Connection count — approaching connection limit causes new connections to fail

LECTURE 19: Content Delivery Networks (CDN)

A Content Delivery Network is a geographically distributed network of servers that delivers content from the node closest to the user. CDNs solve the physics problem: light takes 200ms to travel from India to the US, but it takes only 5ms to travel from Mumbai to a Mumbai CDN server.

1. How CDNs Work

CDN providers maintain 'Points of Presence' (PoPs) — datacenter clusters strategically located in cities and regions worldwide. Cloudflare has 310+ PoPs; AWS CloudFront has 600+; Akamai has 4,200+.

The Request Flow

69. User requests an image: https://example.com/product.jpg
70. DNS resolves to the nearest CDN PoP IP (based on user's geographic location)
71. CDN PoP checks its cache — Cache HIT: return image from local cache (~5ms)
72. Cache MISS: CDN fetches from origin server (your server), caches it, returns it
73. All subsequent users in that region get the cached version

2. What CDNs Cache

Static Assets — Perfect CDN Candidates

- Images (JPEG, PNG, WebP, SVG)
- JavaScript and CSS files
- Video and audio content
- Fonts and icons
- PDF documents

Dynamic Content — Advanced CDN Capability

Modern CDNs (Cloudflare Workers, AWS Lambda@Edge, Vercel Edge Functions) can run compute at the edge, enabling caching or transformation of personalized and dynamic content. This is cutting-edge technology used by Netflix, Shopify, and major media companies.

3. CDN Configuration — Cache-Control Headers

Your origin server instructs the CDN how to cache via HTTP headers:

```
Cache-Control: public, max-age=86400        # Cache for 24 hours
Cache-Control: private, no-store           # Never cache (user-specific data)
Cache-Control: public, max-age=0, must-revalidate  # Cache but always validate
ETag: '33a64df551425fcc55e'               # Fingerprint for conditional requests
```

4. CDN Security Benefits

Beyond performance, CDNs provide security benefits:

- DDoS protection — attack traffic is absorbed by the CDN's massive distributed capacity
- WAF (Web Application Firewall) — block SQL injection, XSS, and common attacks at the edge
- Bot mitigation — detect and challenge automated traffic
- SSL/TLS termination — handle HTTPS at the edge, reducing load on origin servers
- IP reputation filtering — block known bad actors

Interview Insight: For any production system serving global users, a CDN is not optional — it's foundational infrastructure. The cost is modest (a few dollars per terabyte of transfer); the benefit is transformative.

Key Takeaways: CDN = content served from nearest geographic location. Dramatically reduces latency for global users. Cache-Control headers control CDN behavior. Modern CDNs handle dynamic content via edge computing. CDNs also provide DDoS protection and security.

LECTURE 20: Message Queues — Kafka, RabbitMQ & Async Architectures

Message queues enable asynchronous communication between services. Instead of Service A waiting for Service B to respond (synchronous), Service A sends a message to a queue and immediately continues (asynchronous). Service B processes the message when ready.

1. The Core Problem: Tight Coupling

In a tightly coupled synchronous architecture, every downstream service in a chain must be available and fast for the system to work. If email sending takes 2 seconds, the user's order confirmation takes 2 seconds longer than it should. If the inventory system is down, orders can't be placed.

Message queues decouple services. The order service publishes an 'OrderPlaced' message to the queue and immediately returns success. The email service, inventory service, and analytics service all process that message independently, at their own pace.

2. Core Components

Component	Role	Analogy
Producer	Creates and sends messages to the queue	The restaurant taking your order
Broker/Queue	Stores and routes messages	The kitchen order board
Consumer	Reads and processes messages	The chef who makes the food
Message	The data payload sent between components	The order slip
Topic/Channel	Named stream that consumers subscribe to	A specific dish category

3. Kafka vs. RabbitMQ — The Fundamental Choice

RabbitMQ — Traditional Message Queue

RabbitMQ implements the AMQP protocol. Messages are delivered to consumers and deleted after acknowledgment. It's designed for task distribution — one message goes to exactly one consumer.

Use cases: Task queues, job scheduling, work distribution, email sending, report generation.

Apache Kafka — Distributed Event Log

Kafka is fundamentally different. Messages are not deleted after consumption — they're retained in an append-only log for a configurable period (days, weeks, or forever). Multiple consumers can independently read the same message. This enables entirely new architectures.

Use cases: Event sourcing, activity feeds, audit logs, real-time analytics, microservice communication.

Feature	RabbitMQ	Kafka
Message retention	Deleted after ACK	Retained in log (configurable duration)
Consumer model	Competing consumers (one message → one consumer)	Pub/sub (one message → many consumers)
Replay	Not possible	Yes — replay from any point in history
Throughput	High	Extremely high (millions/sec per partition)
Ordering	Per-queue	Per-partition
Use case	Task queues, RPC	Event streaming, microservices, analytics

4. Message Delivery Guarantees

At Most Once

Message is delivered 0 or 1 times. Fast, but messages can be lost on failure. Use for: metrics, analytics where occasional data loss is acceptable.

At Least Once

Message is delivered 1 or more times. Safe from message loss, but consumer may see duplicates. Use for: most business operations. Consumers must be idempotent (Lecture 17).

Exactly Once

Message is delivered exactly once. The hardest guarantee. Requires coordination between producer and consumer. Use for: financial transactions where duplicates are unacceptable.

> ✓ **Pro Tip:** At-least-once delivery + idempotent consumers is the standard production pattern. It's simpler and faster than exactly-once, and idempotency eliminates the harm from duplicate processing.

5. Dead Letter Queues (DLQ)

When a message repeatedly fails processing (bad data format, unrecoverable error), it should not be retried forever. After N retry attempts, move it to a DLQ for manual inspection and remediation.

Every production message queue system should have DLQ configuration. Accumulating failed messages in DLQ without monitoring is a reliability anti-pattern — set alerts when DLQ depth exceeds threshold.

> 💡 **Interview Insight:** In system design interviews, always mention DLQs when discussing message queues. It shows you've thought about failure handling — a sign of production experience.

Key Takeaways: Message queues decouple producers and consumers. Kafka is a distributed event log; RabbitMQ is a traditional message queue. At-least-once delivery + idempotent consumers is the standard pattern. Always configure Dead Letter Queues for failed message handling.

LECTURE 21: Event-Driven Architecture

Event-driven architecture (EDA) is a paradigm where components communicate by producing and consuming events. An event is a fact — something that happened in your system. 'OrderPlaced', 'PaymentCompleted', 'UserRegistered'. Events are broadcast to all interested parties; no central coordinator decides who reacts.

1. Events vs. Commands

This distinction is fundamental and often confused:

Concept	Definition	Example	Direction
Command	A request to do something	ProcessPayment	One sender → one specific receiver
Event	A notification that something happened	PaymentProcessed	One sender → all interested parties

Commands are imperative — 'do this.' Events are declarative — 'this happened.' Events don't know or care who receives them; they simply announce a fact.

2. Pub/Sub Pattern

Publishers emit events to topics without knowing who's subscribed. Subscribers declare interest in topics and receive all events on those topics. This enables complete decoupling — adding a new subscriber requires zero changes to the publisher.

Example: User registration flow with EDA

- Auth Service emits 'UserRegistered' event
- Email Service subscribes → sends welcome email
- Analytics Service subscribes → records new user metric
- Recommendation Service subscribes → initializes user preferences
- Bonus Service subscribes → credits welcome bonus

Each of these services can be added, modified, or removed independently. The Auth Service never changes. This is the power of event-driven architecture.

3. Event Sourcing — Advanced Pattern

Instead of storing the current state, event sourcing stores the sequence of events that led to the current state. The current state is derived by replaying events.

Example: Bank account as event stream

```
AccountOpened: {balance: 0}
MoneyDeposited: {amount: 50000}
```

```
MoneyWithdrawn: {amount: 20000}
MoneyDeposited: {amount: 10000}
```

Current balance: 40,000. Computed by replaying events. This approach provides a complete audit trail, enables time-travel debugging, and allows rebuilding system state from scratch.

> ⚠ **Warning:** Event sourcing adds significant complexity. Use it when you have genuine requirements for audit trails, temporal queries, or rebuilding state — not as a default architecture choice.

4. Challenges in EDA

- Eventual consistency — events propagate asynchronously; different parts of the system are temporarily out of sync
- Debugging complexity — tracing a request through a series of events requires distributed tracing tooling
- Duplicate events — at-least-once delivery means consumers see events multiple times; idempotency is mandatory
- Event ordering — events from the same producer maintain order; events from different producers may arrive out of order
- Schema evolution — events are long-lived; changing event format requires backward compatibility

LECTURE 22: Background Jobs & Worker Systems

Background jobs are tasks that run outside the user's request cycle. When a user clicks 'Upload Report', they shouldn't wait 30 seconds for the report to generate. Instead: accept the request instantly, queue the generation job, and notify the user when ready.

1. Job Types and Use Cases

Job Type	Description	Examples
Fire-and-Forget	Execute once, no result needed	Send welcome email, log analytics event
Scheduled (Cron)	Execute on a schedule	Daily report, hourly data sync, midnight cleanup
Delayed	Execute after a time delay	Send reminder 24 hours after signup, retry payment in 1 hour
Batch	Process large volumes in bulk	Generate invoices for 10,000 users, ETL data pipeline
Long-running	May take minutes or hours	Video transcoding, ML model training, PDF generation
Polling	Periodically check for conditions	Order status update, third-party API sync

2. Job Queue Architecture

Producer (API Server)

When a user action triggers a background task, the API server creates a job descriptor — the task to execute and its parameters — and enqueues it.

```
// .NET with Hangfire
BackgroundJob.Enqueue(() => EmailService.SendWelcomeEmail(userId));
BackgroundJob.Schedule(() => ReminderService.SendReminder(userId), TimeSpan.FromDays(1));
```

Queue (Broker)

The queue persists the job until a worker picks it up. Must be durable — if the queue server restarts, jobs must survive. Redis, RabbitMQ, and dedicated job queue systems (Hangfire, Sidekiq, Celery) all provide durable persistence.

Worker (Consumer)

Workers are long-running processes that continuously poll the queue for new jobs. When a job appears, a worker claims it, executes it, and acknowledges completion. Multiple workers process jobs in parallel, providing horizontal scaling.

3. Critical Worker Design Considerations

Idempotency

Workers must be idempotent. If a worker crashes mid-job and the job is retried, processing should not cause duplicate effects. Check: has this job already been completed? If yes, skip it.

Poison Pills

A poison pill is a job that always fails — bad data format, external service permanently down, code bug. Without handling, this job will be retried forever, blocking the queue. After N retries, move to a Dead Letter Queue for manual inspection.

Visibility Timeout

When a worker claims a job, the queue hides the job for a 'visibility timeout' period. If the worker doesn't complete and acknowledge the job within this period, the queue makes the job visible again for another worker to claim. Set visibility timeout longer than the expected job execution time.

> **Pro Tip:** In .NET, Hangfire is the dominant background job library. It supports all job types, has a built-in dashboard for monitoring, supports multiple storage backends (SQL Server, Redis), and integrates cleanly with ASP.NET Core.

LECTURE 23: Service Discovery

In microservices, services need to find each other to communicate. In a static system, you could hardcode IP addresses. In a dynamic cloud environment — where containers restart, services scale up and down, and IPs change constantly — hardcoded addresses fail immediately.

1. The Dynamic Environment Problem

In Kubernetes, when a pod (container) restarts, it gets a new IP address. When your Payment Service horizontally scales from 3 to 10 instances, 7 new IPs appear. When a service is redeployed, all old IPs disappear and new ones appear.

How does your Order Service know where to send a request to the Payment Service? It can't hardcode IPs — they change constantly. It needs dynamic service discovery.

> ⍰ **Real-World Analogy:** Think of a company directory that updates in real time. As employees join, leave, and change departments, the directory reflects the current state. When you need to reach the payments team, you check the directory for their current contact, not a business card from last year.

2. Client-Side Discovery

The client queries a service registry to find available instances of the target service, then uses a client-side load balancing algorithm to choose which instance to call.

Flow

74. Payment Service starts, registers itself: {name: 'payment-service', ip: '10.0.1.5', port: 8080, status: healthy}
75. Order Service needs to call Payment Service
76. Order Service queries registry: 'Give me all healthy instances of payment-service'
77. Registry returns list of instances
78. Order Service picks one (round-robin, random, etc.) and calls it directly

Tools: Netflix Eureka, HashiCorp Consul

Advantage: Client has full control over load balancing logic.

Disadvantage: Every service must implement discovery logic. Couples your services to the discovery mechanism.

3. Server-Side Discovery (Kubernetes Model)

The client calls a load balancer or API gateway without knowing about specific instances. The load balancer queries the service registry and forwards the request.

This is how Kubernetes Services work. You call 'http://payment-service/' and Kubernetes DNS resolves this to the ClusterIP of the Service, which load balances across all healthy pod IPs.

Advantage: Discovery logic is infrastructure concern, not service concern. Services don't need discovery code.

Disadvantage: Extra network hop through load balancer.

4. Health Checking — Keeping the Registry Accurate

The service registry is only useful if it accurately reflects which services are healthy. Health checking keeps the registry current:

- Active health checks — registry pings each service periodically; removes unhealthy services
- Passive health checks — registry monitors actual traffic; removes services with high error rates
- Heartbeat — services actively send 'I'm alive' signals; if signals stop, remove from registry

TTL-based registration: services must re-register (heartbeat) every N seconds. If they fail to do so, registration expires and they're removed from the registry automatically.

LECTURE 24: API Gateway Architecture

The API Gateway is the single entry point to your backend systems. Instead of clients making separate calls to multiple microservices, they make one call to the gateway, which routes, aggregates, transforms, and secures the interaction.

1. The Chaos Without a Gateway

Imagine a mobile app that needs to display a user's profile: name, recent orders, account balance, and recommendations. Without a gateway, the app must:

79. Call User Service for profile data
80. Call Order Service for recent orders
81. Call Payment Service for balance
82. Call Recommendation Service for suggestions

Four separate HTTP calls, four separate authentication checks, four separate network round trips. The app code is complex, the latency adds up, and every service must implement auth independently.

With an API Gateway: one call to /profile aggregates all this data. The gateway calls the four services internally (in parallel), combines the results, and returns them in a single response.

2. API Gateway Responsibilities

Request Routing

The gateway routes requests to appropriate backend services based on URL path, headers, query parameters, or HTTP method.

```
GET /api/users/101  →  User Service
GET /api/orders     →  Order Service
POST /api/payments  →  Payment Service
```

Authentication & Authorization

Centralized auth: the gateway validates JWT tokens on every request. Only valid, authorized requests reach backend services. Services can trust requests from the gateway without re-validating credentials.

Rate Limiting

The gateway enforces rate limits before requests reach services, protecting backends from overload.

Request Transformation

Transform requests for protocol compatibility: convert REST to gRPC, add required headers, modify request format for legacy backends.

Response Aggregation

Combine responses from multiple services into a single client response. Reduces client complexity and network overhead.

Circuit Breaking

If a backend service is failing, the gateway can circuit-break it — returning cached responses or graceful fallbacks rather than propagating failures to clients.

3. Backend for Frontend (BFF) Pattern

Different clients (mobile, web, admin) have different data requirements. A mobile app needs compact responses to save bandwidth; a web app can handle more data; an admin dashboard needs additional fields.

The BFF pattern creates specialized gateways for each client type: a Mobile BFF optimized for mobile responses, a Web BFF for web clients. Each BFF can aggregate and transform data specifically for its client.

> ⍰ **Interview Insight:** API Gateway is one of the most common topics in system design interviews. Know: what it does (routing, auth, rate limiting, aggregation), the BFF pattern, and how it differs from a load balancer (smarter, application-aware).

LECTURE 25: Microservices Architecture — Deep Dive

Microservices architecture structures an application as a collection of small, independently deployable services, each responsible for a specific business capability. It's the dominant architecture pattern for large-scale systems at companies like Netflix, Amazon, Uber, and Spotify.

1. The Driving Forces Behind Microservices

Large monolithic applications hit walls:

- Team scaling — hundreds of developers can't work in one codebase without constant conflicts
- Technology lock-in — the whole app uses the same language, framework, and database
- Deployment coupling — changing a payment feature requires deploying the entire application
- Selective scaling — can't scale just the search feature; must scale everything
- Single point of failure — a bug in the recommendations module can crash the checkout flow

Microservices solve these by decomposing along business capability boundaries.

2. Core Microservices Principles

Single Responsibility

Each service does one thing and does it well. 'User Service' manages user accounts. 'Payment Service' processes payments. 'Notification Service' sends emails and SMS. When the scope is small, the service is easy to understand, test, and change.

Database per Service

Each service owns its data and its database. No service can directly access another service's database — this would create tight coupling and make independent deployment impossible. If Service A needs data from Service B, it calls Service B's API.

> ⚠ **Warning:** Sharing a database between microservices is a common anti-pattern that creates invisible coupling. If two services share a table, they can't be deployed independently — any schema change must coordinate both services.

Failure Isolation

If the Recommendation Service fails, it doesn't take down Order Service, Payment Service, or User Service. Each service has its own process, its own memory, and its own failure domain.

3. Inter-Service Communication

Pattern	Use When	Tradeoff
REST HTTP (synchronous)	Real-time response required (user-facing APIs)	Tight coupling; both services must be available
gRPC (synchronous)	High-performance, strongly typed service-to-service calls	Protocol buffers required; binary protocol
Message Queue (async)	Decoupling needed; response not immediately required	Eventual consistency; more complex error handling
Event streaming (async)	Multiple consumers; event replay needed	Eventual consistency; significant infrastructure

4. Microservices Challenges — The Honest Tradeoffs

Distributed Systems Complexity

Microservices inherit all distributed systems problems: network failures, latency, data consistency, service discovery, distributed tracing. What was a function call is now a network request with all its associated failure modes.

Data Consistency

With a database per service, multi-service operations are distributed transactions. Maintaining data consistency across services requires Saga patterns (Lecture 27) — significantly more complex than a single-database ACID transaction.

Operational Overhead

10 microservices means 10 deployment pipelines, 10 monitoring configurations, 10 sets of logs to aggregate, 10 sets of dependencies to update. The operational cost is real and often underestimated.

> ⍰ **Interview Insight:** The best microservices advice: start as a well-structured monolith, identify natural service boundaries as the system grows, and extract services when the monolith becomes genuinely painful. Premature microservices are as harmful as premature optimization.

LECTURE 26: Monolith vs. Microservices — Making the Right Choice

The microservices vs. monolith debate is often framed as an evolution — monolith first, microservices later. But there are genuine reasons to choose each, and the wrong choice at the wrong scale creates enormous problems.

The Monolith Advantages — Often Underrated

- Simple development — one codebase, one IDE, one deployment
- Easy debugging — stack traces make sense; no distributed tracing needed
- Atomic deployments — everything changes together; no API versioning needed
- Simple testing — integration tests are local; no test environment complexity
- Lower latency — function calls instead of network calls between components
- Easier data consistency — one database, one transaction

The Microservices Advantages — When You Need Them

- Independent scaling — scale only the services under load
- Technology diversity — different services can use different languages and databases
- Independent deployment — update payment without redeploying the entire app
- Team autonomy — teams own services end-to-end without coordinating deployments
- Fault isolation — one service fails without cascading

The Decision Framework

Factor	Choose Monolith	Consider Microservices
Team size	< 10 engineers	> 50 engineers
Application maturity	Early startup, finding product-market fit	Established domain boundaries
Scaling requirements	Uniform load, simple scaling needs	Components with vastly different scaling needs
Deployment frequency	Weekly or less	Multiple times per day per service
Domain complexity	Simple CRUD, limited business logic	Complex, well-understood bounded contexts
Operational maturity	Small ops team, limited DevOps	Strong DevOps, container orchestration

The Strangler Fig Pattern — Migrating From Monolith

When a monolith must evolve to microservices, the safest approach is the Strangler Fig Pattern (named after a tree that grows around a host tree and eventually replaces it):

83. Keep the monolith running — no big-bang rewrite
84. Identify one bounded context to extract first (typically authentication or a non-critical feature)
85. Build the new microservice alongside the monolith
86. Route traffic for that feature to the new service via API gateway
87. Once stable, remove the corresponding code from the monolith
88. Repeat for the next service

This approach reduces risk dramatically. You're never without a working system.

LECTURE 27: Distributed Transactions — Saga Pattern & 2PC

In a single database, transactions are atomic — either all operations succeed or all fail. In microservices, a 'transaction' might span 5 services and 5 databases. Maintaining atomicity across distributed systems is one of the hardest problems in system design.

1. The Distributed Transaction Problem

User places an e-commerce order:

89. Order Service: create order (SUCCESS)
90. Payment Service: charge credit card (SUCCESS)
91. Inventory Service: reserve items (FAILURE — out of stock)

Payment has been charged but the order can't be fulfilled. We need to refund the payment. But the Payment Service has already committed its transaction. We need a mechanism to coordinate rollback across distributed services.

2. Two-Phase Commit (2PC) — The Classical Approach

A coordinator manages a global transaction across all participating services.

Phase 1: Prepare

Coordinator asks each participant: 'Can you commit?' Each participant:

- Executes the operation but doesn't commit
- Writes to a transaction log
- Responds YES (can commit) or NO (cannot commit)

Phase 2: Commit or Abort

If ALL participants respond YES: coordinator sends COMMIT to all. If ANY respond NO: coordinator sends ABORT to all, triggering rollback.

Problems with 2PC:

- Blocking — if the coordinator crashes after Phase 1, all participants are stuck holding locks
- Performance — two network round trips for every write operation
- Availability — if any participant is unavailable, the transaction blocks

3. Saga Pattern — The Modern Approach

Sagas model long-running distributed transactions as a sequence of local transactions, each with a compensating transaction that undoes its effects if a later step fails.

The e-commerce example with Saga:

Forward Path (Happy Path)

```
Step 1: Order Service → CreateOrder → Success → emit OrderCreated event
Step 2: Payment Service → ProcessPayment → Success → emit PaymentProcessed event
Step 3: Inventory Service → ReserveItems → FAILURE
```

Compensating Transactions (Rollback)

```
Compensation 2: Payment Service → RefundPayment (undo Step 2)
Compensation 1: Order Service → CancelOrder (undo Step 1)
```

Each step is a local transaction. If a step fails, the saga executes compensating transactions in reverse order to restore consistency.

4. Saga Implementation Styles

Choreography

Services react to events from other services. No central coordinator. Each service knows what to do when it receives an event. Decentralized, but can be hard to track the overall flow.

Orchestration

A central Saga Orchestrator coordinates all steps. It tells each service what to do and when. Easier to understand and monitor, but introduces a coordination dependency.

> ✅ **Pro Tip:** For simple sagas (3-4 steps), choreography is clean. For complex sagas (5+ steps with complex failure scenarios), orchestration is easier to reason about and debug.

> 🎯 **Interview Insight:** Saga is the answer to 'How do you handle distributed transactions in microservices?' in interviews. Know both choreography and orchestration, understand compensating transactions, and mention that eventual consistency is the tradeoff.

LECTURE 28: Logging Systems — From Files to Distributed Log Management

Logs are your system's memory. When something goes wrong at 2 AM, logs are how you reconstruct what happened, why it happened, and how to prevent it from happening again. At scale, log management becomes its own engineering challenge.

1. Log Levels — A Discipline

Using the right log level is not just convention — it determines whether your logs are actionable. Log levels, from least to most severe:

Level	Use When	Example
TRACE/VERBOSE	Extremely detailed debugging (dev only)	Entering method CalculateTax(), arg: {amount: 1000}
DEBUG	Diagnostic information (dev/staging)	Cache miss for key user:101, fetching from DB
INFO	Normal operational events	Order 5001 created successfully for user 101
WARN	Something unusual but recoverable	Response time 3s exceeds threshold, DB latency elevated
ERROR	A failure occurred, needs investigation	Payment processing failed: gateway timeout after 30s
CRITICAL/FATAL	System is in an unrecoverable state	Database connection pool exhausted, service shutting down

> ⚠ **Warning:** Never log passwords, API keys, credit card numbers, OTPs, or personal identifiable information (PII). Log files are often stored insecurely and accessible to many people. This is a GDPR and security requirement.

2. Structured Logging — The Production Standard

Plain text logs are human-readable but machine-unfriendly:

```
[ERROR] Payment failed for user 101 at 10:32:45
```

Structured logs (JSON) are machine-parseable, searchable, and filterable:

```
{"level": "ERROR", "timestamp": "2026-03-24T10:32:45Z", "service": "payment-service",
"userId": 101, "orderId": 5001, "error": "Gateway timeout", "duration_ms": 30014}
```

With structured logs, you can answer: 'Show me all ERROR logs from the payment service in the last hour for users in Mumbai.' Plain text logs cannot support this.

3. Centralized Logging in Microservices

With 20 microservices each writing logs to their own files on their own servers, debugging a distributed request is impossible. You need centralized log aggregation.

The ELK Stack (Elasticsearch, Logstash, Kibana)

- Services write structured logs to stdout/stderr
- Log shipper (Filebeat, Fluentd) collects logs from all services
- Logstash processes, transforms, and forwards logs
- Elasticsearch indexes all logs, enabling fast full-text search
- Kibana provides a web interface for searching, visualizing, and dashboarding logs

4. Correlation IDs — Tracing Requests Across Services

When user 101's request to GET /orders fails, logs show errors in Order Service. But those errors were caused by a failure in Inventory Service, triggered by a timeout in a database called by Product Service.

Without correlation IDs, you'd never connect these events. With correlation IDs:

92. API Gateway generates a unique request ID: X-Request-ID: abc-123
93. Gateway passes this ID to all downstream service calls as a header
94. Every service includes this ID in every log entry it writes
95. When you search logs for 'abc-123', you see the complete story across all services

```
{"requestId": "abc-123", "service": "order-service", "event": "Processing order"}
{"requestId": "abc-123", "service": "inventory-service", "event": "DB timeout"}
```

LECTURE 29: Monitoring & Metrics — Observability at Scale

If logging is your system's memory, monitoring is your system's nervous system — continuously sensing the current state and alerting when something is wrong. Without monitoring, you learn about problems from users, not systems.

1. The Three Pillars of Observability

Pillar	What it provides	Tools
Metrics	Numerical measurements over time (CPU, RPS, error rate)	Prometheus, Grafana, Datadog
Logs	Detailed records of specific events	ELK Stack, Splunk, Grafana Loki
Traces	End-to-end request journey across services	Jaeger, Zipkin, AWS X-Ray

True observability requires all three. Metrics tell you something is wrong. Logs tell you what happened. Traces show you where in the distributed system it happened.

2. Google's Four Golden Signals

Google's Site Reliability Engineering (SRE) team identified four signals that, if monitored, provide comprehensive visibility into service health:

Latency

How long it takes to serve requests. Separately track latency of successful requests and failed requests — a spike in error rate with low latency suggests fast failures; slow failures suggest upstream dependencies.

Traffic

How much demand is being placed on your system. Measured as requests per second, queries per second, or writes per second depending on the service type.

Errors

The rate of requests that fail. Distinguish between explicit failures (HTTP 5xx) and implicit failures (200 response with wrong data). Monitor error rate as a percentage of total traffic.

Saturation

How 'full' your service is. CPU utilization, memory usage, disk I/O, queue depth. Saturation metrics predict future problems — 90% CPU today means 100% and failures tomorrow.

Pro Tip: If you implement monitoring for only these four signals, you'll catch the vast majority of production issues. Start here.

3. Alerting — When to Wake Someone Up

Alerting is the art of knowing when a problem is severe enough to page someone. Too many alerts cause alert fatigue — engineers start ignoring notifications. Too few and real problems go unnoticed.

Actionable Alerts Only

Only alert on conditions that require human action. 'CPU is at 75%' — not alertable, normal operation. 'Error rate > 5%' — alertable, real users are experiencing failures.

Alert on Symptoms, Not Causes

Alert on 'user-visible latency > 1s' rather than 'database CPU > 80%'. Symptoms directly correspond to user impact; causes are one step removed and may have multiple sources.

4. SLOs, SLAs, and Error Budgets

Term	Definition	Example
SLI (Service Level Indicator)	A specific metric you measure	99th percentile latency = 450ms
SLO (Service Level Objective)	Your internal target for an SLI	P99 latency < 500ms for 99.9% of days
SLA (Service Level Agreement)	External commitment to customers	99.9% availability or 10% service credit
Error Budget	Allowed amount of failures (1 - SLO)	0.1% of requests can fail; 43 minutes downtime/month

Error budgets enable data-driven decisions: if you've burned your error budget for the month, freeze risky deployments until the budget resets. If you have budget remaining, invest in new features.

LECTURE 30: Distributed Tracing — Following Requests Across Services

Distributed tracing tracks a single user request as it flows through multiple microservices, capturing timing and metadata at each step. It's the difference between 'something is slow' and 'specifically, the database query in the Inventory Service is taking 2 seconds.'

1. The Visibility Problem in Microservices

In a monolith, a stack trace tells you exactly where something failed. In microservices, a user request might touch 8 services. Logs from each service exist independently. Without tracing, correlating a slow user experience to a specific database query in a specific service is detective work at best, impossible at worst.

2. Core Concepts

Concept	Definition	Example
Trace	The complete journey of a single request	User login request touching Auth, User, Session services
Span	One unit of work within a trace	Auth service JWT validation (20ms)
Trace ID	Unique identifier shared across all spans	abc-def-ghi-123
Span ID	Unique identifier for a single span	xyz-789
Parent Span ID	Links child spans to parent	Allows building the call tree

3. How Tracing Works

96. Request arrives at API Gateway. Gateway generates Trace ID and creates the root span.
97. Gateway propagates Trace ID in request headers to downstream services.
98. Each service creates a child span for its work, recording start time, end time, and metadata.
99. Service-to-service calls propagate the Trace ID and current Span ID.
100. All spans are collected by the tracing backend (Jaeger, Zipkin).
101. Tracing UI assembles spans into a visual waterfall diagram showing the entire request journey.

4. Reading a Trace Waterfall

A trace waterfall shows all spans on a timeline. The horizontal axis is time; each bar is a span. Nested bars show parent-child relationships (a service calling another service).

What to look for:

- Long spans — find the slowest step. This is where to optimize.
- Wide gaps — time between spans may indicate queue wait time or network latency
- Sequential vs parallel — can spans be parallelized? Running calls in parallel reduces total latency

5. OpenTelemetry — The Industry Standard

OpenTelemetry (OTel) is a CNCF project that provides a vendor-neutral API and SDK for generating, collecting, and exporting telemetry data (traces, metrics, and logs). By instrumenting your application with OTel, you can switch between tracing backends (Jaeger, Zipkin, Honeycomb, Datadog) without changing code.

> ✓ **Pro Tip:** OpenTelemetry is the future of observability. Major cloud providers, monitoring tools, and languages all support it. Learn it; it will be the standard for years to come.

> ? **Interview Insight:** Distributed tracing is increasingly a prerequisite for principal engineer roles. Interviewers want to know not just that you'd add logging, but that you understand traces, correlation IDs, and how to diagnose production issues in distributed systems.

LECTURE 31: Containerization with Docker

Containers solve the 'it works on my machine' problem by packaging an application with all its dependencies into a portable, self-contained unit that runs identically on any machine.

1. The Environment Problem

Your application works perfectly in development. It fails in staging. It fails differently in production. Each environment has different OS versions, library versions, system configurations, and runtime settings. Containers eliminate this by making the environment part of the application artifact.

> **Real-World Analogy:** Think of a shipping container. It doesn't matter what's inside — the standardized container can be moved by any ship, train, or truck. The transport mechanism doesn't need to know about the cargo. Similarly, a Docker container can run on any host that has Docker installed — your laptop, a CI server, a cloud VM, or Kubernetes.

2. Virtual Machines vs. Containers

Feature	Virtual Machine	Container
What's virtualized	Entire hardware stack (CPU, RAM, network)	Process isolation (namespace, cgroups)
OS	Each VM has its own full OS	Shares host OS kernel
Size	Gigabytes (OS + dependencies)	Megabytes (just app + libraries)
Startup time	Minutes	Seconds (or milliseconds)
Isolation	Strong (separate kernel)	Good (but shares kernel — less isolated than VMs)
Use case	Full environment isolation, different OS	App packaging, microservices, CI/CD

3. Docker Core Concepts

Dockerfile

A text file with instructions to build a Docker image. Each instruction creates a layer.

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0          # Base image
WORKDIR /app                                       # Set working directory
COPY ./publish .                                   # Copy application files
EXPOSE 80                                          # Document which port the app listens
on
ENTRYPOINT ["dotnet", "MyApp.dll"]                # Command to run when container starts
```

Image

A Docker image is the blueprint — an immutable, layered file system containing your application and its dependencies. Images are built from Dockerfiles and stored in registries (Docker Hub, AWS ECR, Azure Container Registry).

Container

A running instance of a Docker image. Like an object instantiated from a class. Multiple containers can run from the same image simultaneously.

4. Docker Compose — Multi-Container Development

Real applications aren't a single container — they're an API server, a database, a cache, and perhaps a message queue. Docker Compose defines and runs multi-container applications.

```
services:
  api:
    build: .
    ports: ['5000:80']
    environment:
      - DB_CONNECTION=Server=db;Database=myapp
  db:
    image: postgres:16
    volumes: ['pgdata:/var/lib/postgresql/data']
  redis:
    image: redis:7-alpine
```

docker-compose up brings your entire development environment up with one command. This is standard practice for local development.

5. Best Practices

- Use minimal base images — Alpine Linux-based images are 5-10x smaller than Ubuntu-based
- Multi-stage builds — use one stage for building, another for the final image, keeping the image small
- Don't run as root — create a non-root user in your Dockerfile for security
- Use .dockerignore — exclude unnecessary files (node_modules, .git, test files) from the build context
- Pin image versions — don't use 'latest' tag in production; pin to specific version like postgres:16.2

LECTURE 32: Kubernetes Architecture

Kubernetes (K8s) is a container orchestration platform. It manages the lifecycle of containers at scale — deploying them, scaling them, self-healing them, and routing traffic to them. If Docker answers 'how do I package and run one container?', Kubernetes answers 'how do I manage 1,000 containers across 100 servers?'

1. The Problem Kubernetes Solves

Docker runs containers. But in production:

- Who restarts containers when they crash?
- How do you distribute containers across 50 servers?
- How do you roll out a new version without downtime?
- How do you scale from 3 to 30 container instances under load?
- How do services find each other when containers move between servers?

Kubernetes answers all of these questions.

2. Kubernetes Architecture

Control Plane (The Brain)

Component	Role
API Server	The front door to Kubernetes. All kubectl commands and internal components communicate through it.
etcd	Distributed key-value store. The cluster's source of truth — stores all cluster state.
Scheduler	Decides which worker node each pod runs on, considering resource availability and constraints.
Controller Manager	Continuously reconciles desired state with actual state. Creates missing pods, removes excess ones.

Worker Nodes

Component	Role
Kubelet	Agent on each node. Ensures containers in pods are running and healthy.
Container Runtime	Runs containers (Docker, containerd, CRI-O).
Kube-proxy	Maintains network rules for service routing. Routes traffic to the correct pod.

3. Core Kubernetes Objects

Pod

The smallest deployable unit. A Pod contains one or more containers that share network namespace and storage. Pods are ephemeral — they can be created, killed, and replaced. Never rely on a specific Pod's IP address or hostname.

Deployment

Manages the lifecycle of Pods at scale. Defines desired state (e.g., '3 replicas of my-api') and continuously reconciles actual state to match. Handles rolling updates and rollbacks.

Service

A stable network endpoint for a set of Pods. Even as Pods die and restart with new IPs, the Service provides a consistent DNS name and IP. Services load balance traffic across all healthy Pods.

ConfigMap and Secret

Decouple configuration from container images. ConfigMaps store non-sensitive configuration; Secrets store sensitive data (passwords, API keys) with base64 encoding (and should be encrypted at rest).

4. Kubernetes Self-Healing

Kubernetes continuously monitors the desired state (defined in YAML manifests) against actual state. If a Pod crashes, the controller creates a new one. If a node fails, Pods are rescheduled to other nodes. This happens automatically, without human intervention.

> **Pro Tip:** Kubernetes self-healing is only as good as your health checks. Configure liveness probes (is the container alive?) and readiness probes (is the container ready to serve traffic?) correctly.

5. Horizontal Pod Autoscaling (HPA)

HPA automatically scales the number of Pod replicas based on observed CPU utilization, memory usage, or custom metrics. When traffic spikes, HPA scales out; when traffic decreases, it scales in.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    name: my-api
  minReplicas: 3
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        averageUtilization: 70
```

LECTURE 34: Deployment Strategies — Blue-Green, Canary & Feature Flags

How you deploy matters as much as what you deploy. A new version with better features is useless if the deployment causes an outage. Modern deployment strategies enable continuous delivery of software with minimal risk.

1. Blue-Green Deployment

Maintain two identical production environments: Blue (current) and Green (new version). When ready to release:

102. Deploy new version to the Green environment
103. Run tests and verification against Green (without serving production traffic)
104. Switch traffic from Blue to Green via load balancer or DNS
105. Monitor Green carefully
106. If issues arise: instantly switch traffic back to Blue

Blue becomes the standby; Green becomes active. On the next release, the roles reverse.

Key advantage: instant rollback. If anything goes wrong, flip the switch back.

Key disadvantage: requires double infrastructure (two complete production environments).

2. Canary Deployment

Release the new version to a small percentage of users first. Monitor metrics carefully. If everything looks good, gradually increase the percentage.

```
Initial: 1% of users → new version
After 30 minutes with no issues: 10% → new version
After 2 hours: 50% → new version
After 24 hours: 100% → new version
```

This strategy limits blast radius. If the new version has a bug, only 1% of users are affected — not all of them. The 'canary' metaphor comes from coal miners who used canaries to detect toxic gas — the canary (small user group) detects problems before they affect everyone.

3. Feature Flags

Deploy code without enabling features. Features are controlled by configuration flags that can be toggled without deployment. This decouples release (merging code) from activation (enabling the feature).

Use Cases

- Dark launch — deploy a new feature and test it with internal users before enabling for customers
- A/B testing — enable feature for 50% of users to measure impact on metrics
- Gradual rollout — start with 1% of users, increase as confidence grows
- Kill switch — instantly disable a problematic feature without rollback

Feature flag platforms: LaunchDarkly, Unleash, AWS AppConfig, custom Redis-backed implementation.

4. Rolling Deployment

Update servers one at a time (or in small batches), with the new version running alongside the old version during the transition. Default deployment strategy in Kubernetes.

Kubernetes rolling update: stop 1 old pod, start 1 new pod. Wait for new pod to pass health checks. Repeat. Traffic is always being served — no downtime.

> ⚠ **Warning:** During a rolling deployment, both old and new versions run simultaneously. Your API must support clients using both versions (backward compatibility). This is especially important for database schema changes — new code must work with both old and new schema during the transition.

> 💡 **Interview Insight:** In interviews about 'zero-downtime deployments', discuss these strategies. Blue-green for simple switches with instant rollback, canary for risk reduction with gradual rollout, feature flags for decoupling deployment from activation.

LECTURE 35: Case Study — Designing a URL Shortener

URL shortener design is one of the most common system design interview questions. It appears simple on the surface but reveals deep understanding of scalability, database design, caching, and distributed systems when explored thoroughly.

1. Requirements Clarification

Before designing anything, clarify requirements. This discipline separates strong system design candidates from weak ones.

Functional Requirements

- Given a long URL, generate a short URL (e.g., short.ly/abc123)
- Redirect short URLs to the original long URL
- Short URLs must be unique globally
- Optional: custom short URLs, expiration dates, click analytics

Non-Functional Requirements

- Scale: 100 million URLs created per day, 10 billion redirects per day
- Redirect latency: < 50ms at P99
- Availability: 99.99% (52 minutes downtime per year)
- URL uniqueness: no collisions

2. Scale Estimation

100 million URLs created per day = 1,160 creates/second (peak ~5,000/second)

10 billion redirects per day = 115,000 redirects/second (peak ~500,000/second)

Read:Write ratio = 100:1 — massively read-heavy

Storage: 100M URLs/day × 365 × 5 years × ~500 bytes/record = ~90 TB over 5 years

3. Short Code Generation Strategies

Base62 Encoding of Auto-Increment ID

Assign each URL an auto-incrementing integer ID. Encode that ID in Base62 (characters a-z, A-Z, 0-9). ID 125 encodes to '3R'. A 7-character Base62 code supports 62^7 = 3.5 trillion unique codes.

Advantages: Guaranteed uniqueness (no collision checking), predictable growth.

Disadvantages: Predictable — users can guess adjacent URLs (security concern). Single sequence generator is a bottleneck.

Random Code + Collision Check

Generate a random 7-character string. Check if it already exists in the database. If it does, regenerate.

Disadvantages: Collision probability, requires DB round-trip for uniqueness check.

Pre-Generated Code Pool — Production Best Practice

Pre-generate millions of unique codes and store them in a 'codes available' table. When a URL is created, claim one code atomically. No collision risk, no sequential enumeration, fast allocation.

> **Pro Tip:** The pre-generated code pool approach is used in production by companies like Bitly. It eliminates collision checking at creation time and makes the code generation process simple and reliable.

4. Database Design

```
CREATE TABLE urls (
  id           BIGINT PRIMARY KEY,
  short_code   VARCHAR(10) UNIQUE NOT NULL,   -- indexed for fast redirect lookup
  long_url     TEXT NOT NULL,
  user_id      BIGINT,                        -- NULL for anonymous
  created_at   TIMESTAMP DEFAULT NOW(),
  expires_at   TIMESTAMP,                     -- NULL = never expires
  click_count  BIGINT DEFAULT 0,
  INDEX idx_short_code (short_code)           -- CRITICAL for redirect performance
);
```

5. High-Level Architecture

Component	Purpose	Technology
API Layer	Handle create and redirect requests	Load-balanced application servers
Redis Cache	Cache short_code → long_url mappings	Redis cluster with LRU eviction
Primary Database	Source of truth for URL mappings	PostgreSQL with sharding
Read Replicas	Handle read-heavy redirect lookup	Multiple PostgreSQL replicas
CDN	Cache redirect responses at edge	Cloudflare / AWS CloudFront
Analytics Pipeline	Process click events asynchronously	Kafka + streaming processor

6. The Redirect Flow

107. User requests: GET short.ly/abc123
108. CDN checks cache: MISS (first time for this URL)
109. Request reaches API server
110. Check Redis cache: HIT → return 301 redirect with long URL
111. Cache MISS: query database by short_code index
112. Populate Redis cache (TTL: 24 hours)
113. Return 301 redirect
114. Asynchronously: publish click event to Kafka for analytics

7. Scaling the System

Read Scaling

With a 100:1 read:write ratio, Redis cache is the critical optimization. A well-configured cache with high hit rate (>99% for popular URLs) means the database sees only 1% of redirect traffic.

Write Scaling

5,000 creates/second can be handled by a well-tuned PostgreSQL primary. If write volume grows further, use database sharding (Lecture 8) with short_code as the shard key.

Global Availability

Multi-region deployment with CDN: redirect requests served from edge nodes within 5ms of users worldwide. Origin database updated asynchronously; brief redirect inconsistency is acceptable.

8. Edge Cases and Failure Scenarios

- Viral URL storm — one URL gets 10 million redirects in an hour. CDN absorbs most; Redis handles the rest. Database is shielded.
- Database failure — Redis cache continues serving cached URLs during brief outages. New creates fail gracefully.
- Cache stampede — popular URL expires. Use mutex locking or probabilistic early expiration to prevent database overload.
- Malicious URLs — scan created URLs against blocklists. Rate-limit URL creation per IP/user.

Interview Insight: When presenting URL Shortener in an interview, demonstrate the thought process: requirements → estimation → data model → algorithm → components → scaling → failure handling. This systematic approach is more impressive than any individual technical detail.

Key Takeaways for the URL Shortener: The URL shortener illustrates every major system design concept: database design, indexing, caching, CDN, horizontal scaling, analytics pipelines. The redirect path is aggressively optimized because it's 100x more frequent than URL creation. Cache hit ratio is the north-star metric.

Conclusion — Your System Design Journey Continues

You've covered an enormous amount of ground. From the fundamentals of what a system is, through the mechanics of scalability, databases, caching, and networking, to advanced distributed systems concepts, microservices, and deployment strategies.

What You Now Understand

- How requests travel from browser to server and back — the fundamental plumbing of the internet
- How to scale systems from 100 users to 100 million users — vertical vs. horizontal scaling, load balancing, caching, sharding
- How databases work at scale — SQL vs. NoSQL, ACID, CAP theorem, replication, and sharding
- How distributed systems behave — failure modes, consistency models, and the fallacies that trap engineers
- How modern architectures are built — microservices, API gateways, event-driven design, service discovery
- How systems fail gracefully — fault tolerance, circuit breakers, retries, graceful degradation
- How to deploy safely — blue-green, canary, feature flags, rolling updates
- How to observe systems in production — logging, metrics, distributed tracing

The Most Important Lessons

If you take only three things from this book, take these:

1. Design for Failure

In distributed systems at scale, failure is not exceptional — it is routine. Hardware fails. Networks partition. Services crash. The measure of a well-designed system is not whether failures happen, but how gracefully it handles them when they do.

2. Measure, Then Optimize

Don't guess where your bottlenecks are. Measure with real metrics. Add caching where reads are hot. Add sharding where data is too large. Add workers where processing is slow. Premature optimization is as dangerous as no optimization.

3. Simple First, Complex When Necessary

Start with a monolith. Use a single database. Deploy to one region. Add complexity only when the simplicity hits real limits. Microservices, multi-region deployments, and sophisticated caching strategies all add genuine engineering and operational overhead. That overhead is worth it at scale — but not before.

Continuing Your Learning

System design is a practice, not a destination. The field evolves: new databases emerge, new patterns gain traction, new scale challenges appear. To continue growing:

- Read engineering blogs from Netflix, Uber, LinkedIn, Dropbox, and Discord — they publish detailed accounts of how they solved real scale challenges
- Study the papers — 'Dynamo: Amazon's Highly Available Key-Value Store', Google's MapReduce, Bigtable, and Spanner papers are foundational
- Build things — design a real URL shortener, a message queue, a distributed cache. Implementation reveals what reading obscures.
- Do mock interviews — system design is a skill that requires practice under time pressure
- Follow system design newsletters and communities — the field moves fast

A Final Note: System design is ultimately about empathy — for users who depend on your system's reliability, for colleagues who maintain it, and for the business it serves. The most technically sophisticated design is worthless if it's too complex to operate, too expensive to run, or too slow to deliver value. Build with purpose.

Build Systems That Matter.